

Chapitre 2 :

Spécifications des Besoins et Étude Technique

Introduction

Ce chapitre a pour but de fournir une vue d'ensemble de notre projet **Bridger**, en précisant les besoins fonctionnels et non fonctionnels, en identifiant les acteurs impliqués dans l'interaction avec le système, en élaborant le backlog produit, et en définissant la planification initiale des releases du projet. Il pose également les fondations techniques en présentant l'architecture, l'écosystème technologique et les outils retenus pour la mise en œuvre.

2.1 Extraction des besoins

La réussite de tout projet de développement logiciel repose avant tout sur une analyse précise et structurée des besoins. Cette étape initiale vise à identifier et formuler les exigences fonctionnelles (ce que l'application doit faire) ainsi que les exigences non fonctionnelles (comment elle doit le faire), afin d'assurer la cohérence, la pertinence et la viabilité de la solution à long terme.

Dans le cadre du projet **Bridger**, cette phase a été menée avec rigueur à partir d'une étude du marché de la livraison collaborative pair-à-pair et de l'analyse des plateformes existantes (Grabr, Worldcraze, PiggyBee). Les besoins ainsi recensés ont ensuite été analysés, affinés et enrichis par nos soins à travers des propositions innovantes — notamment la vérification biométrique faciale par intelligence artificielle, le séquestre Stripe automatisé, la suggestion de prix par modèle ML XGBoost et le suivi temps réel multi-mode (GPS terrestre + tracking aérien OpenSky) — dans le but de renforcer l'expérience utilisateur, de garantir la confiance entre étrangers, et d'assurer une réelle valeur ajoutée par rapport aux solutions existantes sur le marché.

2.2 Les besoins fonctionnels

Avant d'imposer une solution, il faut se tourner vers le demandeur, pour aboutir de manière structurée à la solution. En effet, le but du projet est de satisfaire le besoin. Il faut exprimer clairement les objectifs à atteindre, afin d'éviter toute confusion entre l'équipe de développement et les utilisateurs finaux.

Les acteurs identifiés :

Dans cette section nous allons classer les besoins par acteur. Les acteurs identifiés sont les suivants :

- Les **utilisateurs de l'application** regroupent différents types d'acteurs interchangeables, à savoir :
 - Les **expéditeurs (Senders)** — utilisateurs qui souhaitent envoyer un colis vers une destination donnée ;
 - Les **voyageurs (Travelers)** — utilisateurs qui disposent d'une capacité de transport résiduelle (vol, traversée maritime, voyage routier) et acceptent de transporter le colis d'un expéditeur ;
 - Les **destinataires (Receivers)** — tiers, généralement non inscrits, qui réceptionnent physiquement le colis à l'arrivée ;
- L'**Administrateur** — modérateur disposant de droits étendus sur l'espace d'administration ;

- Le **Système** — acteur logique pour les tâches automatisées (webhooks Stripe, polling OpenSky, watchdog GPS, classifieurs ML).

Besoins fonctionnels identifiés :

Nous avons identifié les besoins fonctionnels de chaque acteur comme suit :

Les utilisateurs de l'application (fonctionnalités communes à tous les utilisateurs inscrits) :

- Visualiser l'écran d'accueil et les écrans d'on-boarding
- S'authentifier
 - S'inscrire via numéro de téléphone et code OTP **Twilio Verify**
 - Vérifier son numéro de téléphone (canal SMS ou WhatsApp)
 - Rafraîchir une session expirée
 - Se déconnecter
- Effectuer la vérification d'identité (KYC biométrique)
 - Téléverser une pièce d'identité (passeport, CIN, permis de conduire)
 - Capturer un selfie soumis aux contrôles de qualité et de vivacité
 - Comparer l'embedding facial à celui du document (similarité cosinus ArcFace)
 - Consulter le statut de la vérification
- Consulter son espace
 - Voir son profil
 - Mettre à jour son profil (avatar, nom, e-mail)
 - Consulter les termes et conditions
- Gérer les notifications
 - Visualiser les notifications push reçues
 - Marquer comme lues / supprimer
 - Configurer les préférences (deals, messages, paiements, promotions)
- Consulter un système de messagerie
 - Visualiser la liste des conversations
 - Envoyer un message texte, image ou de localisation
 - Marquer une conversation comme lue
- Gérer les utilisateurs indésirables
 - Bloquer un utilisateur
 - Signaler un utilisateur (fraude, harcèlement, contenu inapproprié)
- Gérer le portefeuille
 - Recharger le portefeuille via Stripe
 - Consulter l'historique des transactions
 - Effectuer un retrait via Stripe Connect

Les expéditeurs (Senders) — fonctionnalités spécifiques :

- Publier une demande de colis (workflow en cinq étapes guidées)
 - Sélectionner la catégorie, le poids, la taille, la fragilité et la valeur déclarée du colis
 - Définir le trajet de départ et d'arrivée (ville/pays)
 - Saisir les informations du destinataire (nom, téléphone)
 - Définir le prix proposé (avec suggestion ML XGBoost)
 - Réviser et publier l'annonce
- Rechercher un voyageur correspondant à son trajet
- Modifier ou annuler une annonce avant appariement

- Régler le séquestre Stripe une fois apparié à un voyageur
- Suivre la livraison en temps réel
- Recevoir le code à 6 chiffres et le transmettre au destinataire

Les voyageurs (Travelers) — fonctionnalités spécifiques :

- Publier un trajet de voyage
 - Définir l'itinéraire (origine, destination, date)
 - Spécifier le mode de transport (avion, bateau, route)
 - Préciser la capacité maximale (poids, type de colis)
 - Définir le prix de transport (négociable ou fixe)
- Accepter une demande de colis correspondant à son trajet
- Activer le suivi en direct (mode GPS ou mode vol)
- Émettre périodiquement les positions GPS pendant le trajet
- Confirmer le ramassage du colis (vérification du QR code signé)
- S'enregistrer sur Stripe Connect pour recevoir les paiements
- Consulter ses revenus et effectuer un payout

Les destinataires (Receivers) — fonctionnalités spécifiques (sans inscription) :

- Saisir le code de livraison à 6 chiffres
- Confirmer la réception du colis (déclenche la libération du séquestre)

Gestion des avis et litiges (commune aux Senders et Travelers après une transaction) :

- Évaluer un partenaire de transaction (note de 1 à 5 + commentaire)
- Visualiser les avis reçus et leur score de confiance
- Ouvrir un litige sur un Deal terminé ou problématique
 - Choisir un type (item endommagé, perdu, non livré, mauvais article, fraude, autre)
 - Téléverser des preuves (photos, vidéos, documents, texte libre)
 - Échanger des messages avec la partie adverse et un médiateur admin
 - Consulter la chronologie auditable du litige

L'Administrateur :

- S'authentifier via OTP Twilio admin
- Voir les statistiques (tableau de bord avec KPI : utilisateurs, deals, transactions, volumes financiers)
- Vérifier les informations spécifiques des utilisateurs
- Gérer les demandes KYC
 - Examiner les pièces d'identité téléversées
 - Visualiser le score de confiance biométrique
 - Accepter (**APPROVED**) ou refuser (**REJECTED**) la vérification
 - Traiter les cas **MANUAL_REVIEW** issus de la zone de seuil ambiguë
- Gérer les utilisateurs
 - Visualiser la liste complète des utilisateurs
 - Rechercher un utilisateur (par nom, téléphone, e-mail)
 - Flagger / déflagger un compte
 - Bannir / débannir un compte avec saisie de la raison
- Gérer les annonces
 - Visualiser la liste des Shipments (Deals expéditeurs)
 - Visualiser la liste des Trip Posts (annonces voyageurs)
 - Modérer le contenu signalé

- Gérer les transactions et le séquestre
 - Visualiser tous les flux financiers (deposit, escrow, release, refund, payout)
 - Visualiser l'état Stripe Connect des voyageurs
- Gérer les litiges
 - Visualiser tous les litiges (filtrables par statut)
 - Assigner un médiateur
 - Trancher : `FILER_WIN` , `AGAINST_WIN` , `SPLIT`
- Gérer les avis
 - Modérer les avis détectés frauduleux par le classifieur ML
 - Approuver / supprimer un avis flaggé
- Auditer le système
 - Consulter le journal `AuditLog` (action, acteur, IP, horodatage)
 - Filtrer par type d'entité et action
- Gérer les données ML
 - Visualiser et enrichir le jeu de données d'entraînement du modèle de pricing
 - Suivre les métriques de matching et de fraude

Le Système (acteur non humain) :

- Vérifier la validité du numéro de téléphone (`libphonenumber-js`)
- Vérifier la signature des webhooks Stripe (HMAC SHA-256)
- Détecter automatiquement la vivacité d'un selfie (anti-spoofing)
- Extraire automatiquement les informations de la pièce d'identité (OCR multilingue latin + arabe)
- Calculer automatiquement la similarité cosinus entre embeddings ArcFace 512-d
- Suggérer automatiquement un prix de livraison (modèle XGBoost + repli heuristique)
- Détecter automatiquement les avis frauduleux (classifieur XLM-RoBERTa)
- Polling automatique de l'API OpenSky pour le tracking aérien (toutes les 30 s)
- Surveillance automatique (watchdog) des sessions GPS abandonnées
- Envoyer une notification push (FCM/APNs)
- Envoyer un message OTP via Twilio Verify (canal SMS ou WhatsApp)
- Diffuser les événements temps réel via Socket.IO

2.3 Les besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité, de performance et de sécurité que l'application doit respecter.

Pour garantir la fiabilité et la qualité de l'application **Bridger**, les besoins non fonctionnels identifiés incluent :

- **Sécurité**

L'application Bridger doit garantir la sécurité des données personnelles et financières des utilisateurs grâce à une authentification sécurisée par **OTP Twilio Verify**, à des sessions JWT signées HS256, à une protection contre les attaques par force brute (verrou Redis 5 tentatives / 15 min), à une vérification de signature systématique des webhooks Stripe, à des en-têtes HTTP durcis (`helmet`), à un CORS strict et à la signature HMAC des codes QR de livraison.

- **Accessibilité multiplateforme**

Bridger doit être accessible sur **Android** et **iOS** via React Native (Expo SDK 54), ainsi que sur le **web** (espace administrateur en React + Vite), offrant une expérience cohérente et fluide quel que soit le support utilisé.

- **Performance et réactivité**

L'application doit offrir des temps de réponse rapides, une navigation fluide et des fonctionnalités réactives, même avec un grand nombre d'utilisateurs ou des données volumineuses. Les modèles d'IA sont chargés en différé et pré-chauffés au démarrage du microservice ; les colonnes interrogées fréquemment sont indexées (index composites Prisma) ; un chemin de migration vers `pgvector` + index HNSW est documenté pour la recherche vectorielle ANN.

- **Simplicité et ergonomie**

L'interface doit être intuitive, claire et facile à utiliser pour tous les profils d'utilisateurs, qu'il s'agisse d'expéditeurs, de voyageurs ou d'administrateurs. Les flux de publication sont structurés en assistants pas-à-pas (wizards) avec indicateur de progression.

- **Scalabilité**

Bridger doit être conçue pour évoluer facilement et supporter un grand nombre d'utilisateurs sans perte de performance, en tirant parti d'une architecture orientée microservices conteneurisable (Docker), d'un cache Redis distribué, et d'un déploiement cloud sur Azure App Service.

- **Fiabilité et tolérance aux pannes**

Le système doit rester opérationnel même en cas d'erreur ou de panne partielle. Un **Circuit Breaker** isole le backend principal d'une indisponibilité du microservice IA ; un **watchdog** clôt automatiquement les sessions de tracking GPS abandonnées ; les polls de vols actifs sont restaurés après redémarrage ; les erreurs sont remontées et tracées via Sentry.

- **Observabilité**

Le système expose des sondes de santé granulaires (`/health` , `/health/ml` , `/health/face-service`), une journalisation structurée Winston, et un suivi des erreurs avec Sentry (`tracesSampleRate: 0.2` en production).

- **Personnalisation**

L'interface de l'application doit pouvoir s'adapter au profil de l'utilisateur connecté : le store global Zustand bascule entre le rôle Sender et Traveler via le champ `mode` , n'affichant que les fonctionnalités pertinentes. Un mode clair / sombre est également fourni.

- **Confidentialité**

Tous les échanges via le système de messagerie doivent être protégés. Les fichiers KYC sont stockés hors arborescence publique (`UPLOADS_DIR=/home/data/uploads`), les embeddings biométriques sont des vecteurs irréversibles (et non des images), et le code QR de ramassage est signé HMAC-SHA256 avec un secret dédié par Deal.

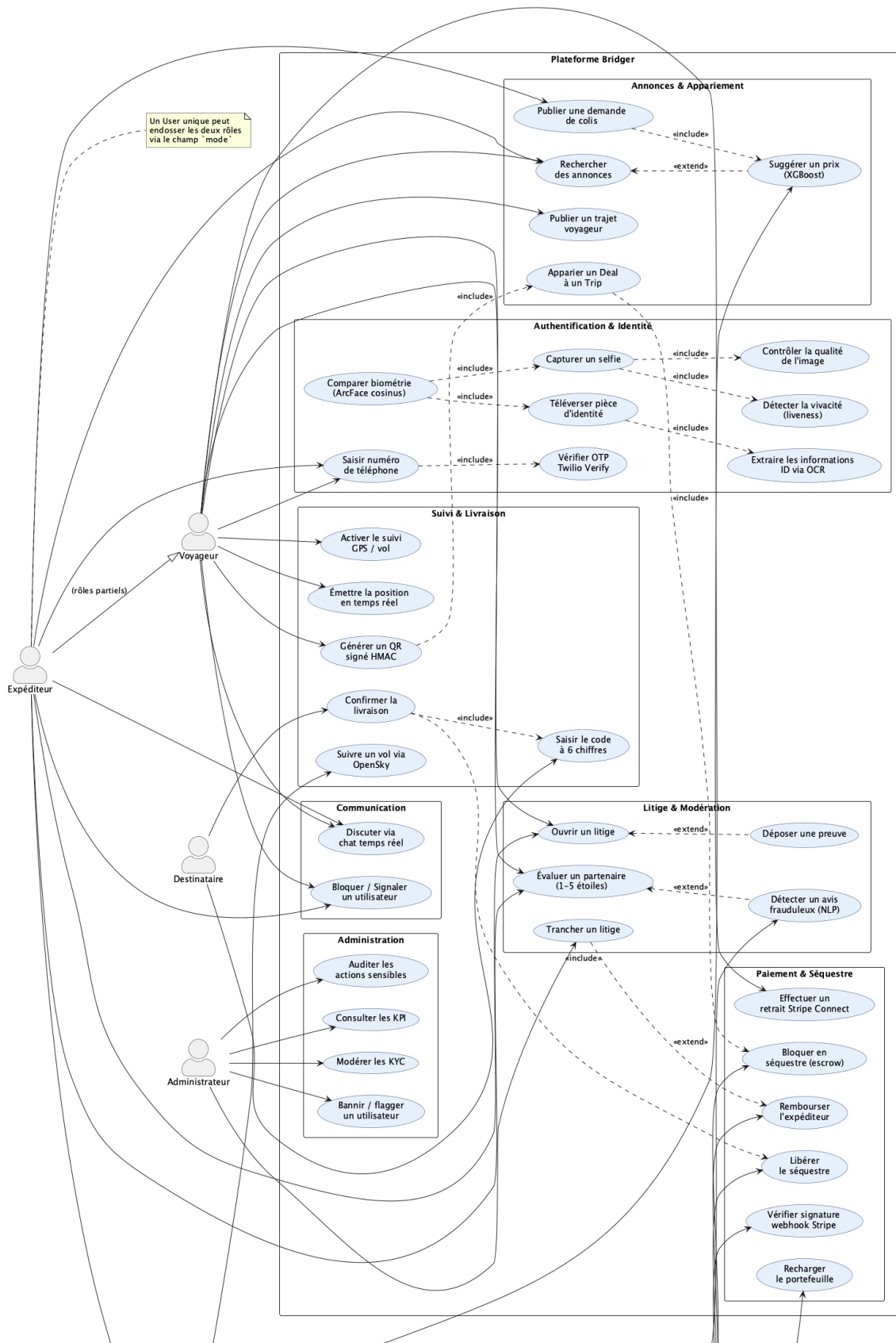
- **Conformité RGPD**

Les données des utilisateurs doivent être traitées conformément au Règlement Général sur la Protection des Données (RGPD), avec consentement explicite, possibilité de suppression complète du compte (`DELETE /users/me cascade onDelete: Cascade`) et transparence sur leur usage.

Un audit trail immuable (`AuditLog`) consigne toutes les actions sensibles avec horodatage et adresse IP.

2.4 Modélisation de la solution (diagramme de cas d'utilisation globale)

Le diagramme de cas d'utilisation est un diagramme UML utilisé pour donner une vision globale du comportement fonctionnel d'un système logiciel.



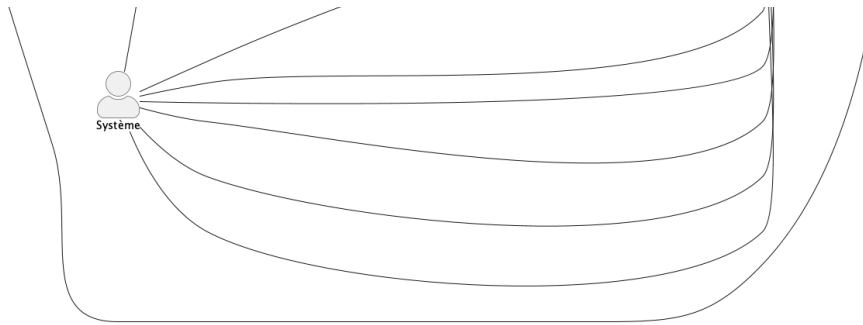


Figure 2.1 : Diagramme de cas d'utilisation globale

Description : Ceci est le diagramme de cas d'utilisation globale qui représente les fonctionnalités reliées à nos acteurs et qui illustre l'utilité de notre application Bridger. Le diagramme distingue sept paquetages fonctionnels (Authentification & Identité, Annonces & Appariement, Paiement & Séquestre, Suivi & Livraison, Communication, Litige & Modération, Administration) qui correspondent à la décomposition des routes de l'API. Les inclusions `<<include>>` traduisent des contraintes métier strictes : par exemple, *Capturer un selfie* inclut obligatoirement *Contrôler la qualité* et *Détecter la vivacité*. Les extensions `<<extend>>` matérialisent les comportements optionnels comme la détection automatique d'un avis frauduleux. Une généralisation lie l'Expéditeur et le Voyageur car un même utilisateur peut endosser les deux rôles via le champ `mode` du store global.

2.5 Product backlog

Le Product Backlog représente la liste priorisée de toutes les fonctionnalités, exigences, améliorations et corrections à apporter à l'application **Bridger**. Il est continuellement mis à jour pour refléter les besoins des utilisateurs (expéditeurs, voyageurs, destinataires et administrateur) ainsi que les objectifs du projet. Chaque élément du backlog décrit une fonctionnalité essentielle, comme l'authentification par OTP Twilio, la vérification biométrique KYC, l'appariement des annonces, le séquestre Stripe, le suivi temps réel ou la résolution de litiges.

ID	Acteur	Fonction globale	ID	User Story	Priorité
1	Utilisateur	Visualiser l'interface d'accueil de l'application	1.1	En tant que nouvel utilisateur, je peux visualiser les écrans d'onboarding afin de découvrir les services proposés.	Élevé
		S'inscrire via téléphone	2.1	En tant que nouvel utilisateur, je peux saisir mon numéro de téléphone et recevoir un code OTP via Twilio Verify (canal SMS ou WhatsApp).	Élevé
		S'authentifier	3.1	En tant qu'utilisateur, je peux saisir le code OTP reçu et obtenir un couple de tokens JWT (access + refresh).	Élevé
		Rafraîchir la session	4.1	En tant qu'utilisateur, je peux rafraîchir mon access token via	Moyenne

				mon refresh token sans me ré-authentifier.	
		Effectuer la vérification KYC	5.1	En tant qu'utilisateur, je peux téléverser ma pièce d'identité (passeport, CIN ou permis de conduire).	Élevé
			5.2	En tant qu'utilisateur, je peux capturer un selfie qui sera soumis à un contrôle de qualité (flou, luminosité, taille du visage).	Élevé
			5.3	En tant qu'utilisateur, mon selfie est soumis à un test de vivacité (anti-spoofing) afin d'empêcher l'utilisation d'une photo.	Élevé
			5.4	En tant qu'utilisateur, mon visage est comparé à celui de ma pièce d'identité par similarité cosinus ArcFace 512-d.	Élevé
			5.5	En tant qu'utilisateur, je peux consulter le statut de ma vérification (PENDING, APPROVED, REJECTED, MANUAL_REVIEW).	Moyenne
		Consulter son profil	6.1	En tant qu'utilisateur, je peux voir et mettre à jour mon profil (avatar, nom, e-mail).	Moyenne
			6.2	En tant qu'utilisateur, je peux consulter les termes et conditions.	Bas
			6.3	En tant qu'utilisateur, je peux me déconnecter et invalider ma session.	Moyenne
		Voir les notifications	7.1	En tant qu'utilisateur, je peux visualiser les notifications push reçues.	Moyenne
			7.2	En tant qu'utilisateur, je peux marquer une notification comme lue ou la supprimer.	Bas
			7.3	En tant qu'utilisateur, je peux configurer mes préférences de notification (deals, messages, paiements, promotions).	Bas

		Consulter un système de messagerie	8.1	En tant qu'utilisateur, je peux accéder à l'interface de messagerie pour consulter mes conversations.	Moyenne
			8.2	En tant qu'utilisateur, je peux envoyer un message texte, image ou de localisation à mon correspondant.	Moyenne
			8.3	En tant qu'utilisateur, je peux marquer une conversation comme lue.	Bas
		Bloquer / Signaler	9.1	En tant qu'utilisateur, je peux bloquer un autre utilisateur.	Bas
			9.2	En tant qu'utilisateur, je peux signaler un utilisateur (fraude, harcèlement, contenu inapproprié).	Moyenne
		Gérer le portefeuille	10.1	En tant qu'utilisateur, je peux recharger mon portefeuille via Stripe (PaymentIntent).	Élevé
			10.2	En tant qu'utilisateur, je peux consulter l'historique de mes transactions.	Moyenne
2	Expéditeur	Publier une demande de colis	11.1	En tant qu'expéditeur, je peux décrire mon colis (catégorie, poids, taille, fragilité, valeur déclarée).	Élevé
			11.2	En tant qu'expéditeur, je peux définir le trajet d'envoi (ville/pays de départ et d'arrivée).	Élevé
			11.3	En tant qu'expéditeur, je peux saisir les informations du destinataire (nom, téléphone).	Élevé
			11.4	En tant qu'expéditeur, je reçois une suggestion de prix issue du modèle ML XGBoost (avec repli heuristique).	Élevé
			11.5	En tant qu'expéditeur, je peux réviser puis publier mon annonce.	Élevé
		Rechercher un voyageur	12.1	En tant qu'expéditeur, je peux rechercher un voyageur	Moyenne

				correspondant à mon trajet.	
		Régler le séquestre	13.1	En tant qu'expéditeur, après appariement, je peux régler le montant qui sera bloqué en séquestre Stripe.	Élevé
		Modifier / Annuler	14.1	En tant qu'expéditeur, je peux modifier ou annuler mon annonce avant appariement.	Moyenne
3	Voyageur	Publier un trajet	15.1	En tant que voyageur, je peux publier un trajet (origine, destination, date, capacité, mode de transport).	Élevé
		Accepter une demande	16.1	En tant que voyageur, je peux accepter une demande de colis correspondant à mon trajet.	Élevé
		Stripe Connect	17.1	En tant que voyageur, je peux m'enregistrer sur Stripe Connect afin de percevoir mes paiements.	Élevé
			17.2	En tant que voyageur, je peux effectuer un retrait (payout) vers mon compte bancaire.	Moyenne
		Activer le suivi temps réel	18.1	En tant que voyageur, je peux activer le suivi GPS terrestre pendant le trajet.	Élevé
			18.2	En tant que voyageur en avion, le système suit automatiquement mon vol via l'API OpenSky.	Élevé
		Confirmer le ramassage	19.1	En tant que voyageur, je peux scanner un QR code signé HMAC pour confirmer le ramassage du colis.	Élevé
4	Destinataire	Confirmer la réception	20.1	En tant que destinataire (sans compte), je peux saisir le code à 6 chiffres pour confirmer la livraison.	Élevé
			20.2	En tant que destinataire, ma confirmation déclenche automatiquement la libération du séquestre vers le voyageur.	Élevé
—	Sender / Traveler	Évaluer un partenaire	21.1	En tant qu'utilisateur, après une transaction, je peux laisser une note de 1 à 5 et un commentaire.	Moyenne

		Ouvrir un litige	22.1	En tant qu'utilisateur, je peux ouvrir un litige sur un Deal terminé ou problématique.	Élevé
			22.2	En tant qu'utilisateur, je peux téléverser des preuves (photos, vidéos, documents).	Élevé
			22.3	En tant qu'utilisateur, je peux échanger des messages avec la partie adverse et le médiateur.	Moyenne
			22.4	En tant qu'utilisateur, je peux consulter la chronologie auditable du litige.	Moyenne
5	Administrateur	S'authentifier	23.1	En tant qu'administrateur, je peux me connecter via OTP Twilio admin et le flag <code>isAdmin</code> .	Élevé
		Voir les statistiques	24.1	En tant qu'administrateur, je peux consulter le tableau de bord (KPI, croissance, taux de matching).	Moyenne
		Vérifier les KYC	25.1	En tant qu'administrateur, je peux examiner les pièces d'identité téléversées et leur score biométrique.	Élevé
			25.2	En tant qu'administrateur, je peux accepter (APPROVED) ou refuser (REJECTED) un dossier KYC.	Élevé
			25.3	En tant qu'administrateur, je peux trancher manuellement les cas <code>MANUAL_REVIEW</code> issus de la zone de seuil ambiguë.	Élevé
		Gérer les utilisateurs	26.1	En tant qu'administrateur, je peux visualiser la liste complète des utilisateurs.	Élevé
			26.2	En tant qu'administrateur, je peux rechercher un utilisateur (par nom, téléphone, e-mail).	Moyenne
			26.3	En tant qu'administrateur, je peux flagger / déflagger un compte.	Moyenne
			26.4	En tant qu'administrateur, je peux bannir / débannir un compte avec saisie de la raison.	Élevé

		Visualiser les annonces	27.1	En tant qu'administrateur, je peux consulter la liste des Shipments (Deals) et Trip Posts.	Élevé
		Visualiser les transactions	28.1	En tant qu'administrateur, je peux consulter tous les flux financiers (deposit, escrow, release, refund, payout).	Élevé
		Gérer les litiges	29.1	En tant qu'administrateur, je peux visualiser tous les litiges et leur SLA de 72 heures.	Élevé
			29.2	En tant qu'administrateur, je peux assigner un médiateur à un litige.	Moyenne
			29.3	En tant qu'administrateur, je peux trancher : FILER_WIN, AGAINST_WIN, SPLIT.	Élevé
		Modérer les avis	30.1	En tant qu'administrateur, je peux examiner les avis détectés frauduleux par le classifieur ML.	Moyenne
			30.2	En tant qu'administrateur, je peux approuver ou supprimer un avis flaggé.	Moyenne
		Auditer le système	31.1	En tant qu'administrateur, je peux consulter le journal AuditLog (action, acteur, IP, horodatage).	Élevé
		Gérer les données ML	32.1	En tant qu'administrateur, je peux visualiser et enrichir le jeu de données d'entraînement du modèle de pricing.	Bas
6	Système	Vérifier le téléphone	33.1	En tant que système, je dois vérifier la validité du numéro de téléphone (libphonenumber-js).	Élevé
		Envoyer l'OTP	34.1	En tant que système, je dois envoyer le code OTP via Twilio Verify (canal SMS ou WhatsApp).	Élevé
		Détecter la vivacité	35.1	En tant que système, je dois détecter automatiquement la vivacité d'un selfie (anti-spoofing).	Élevé
		Extraire les info ID	36.1	En tant que système, je dois extraire automatiquement les informations de la pièce	Élevé

				d'identité par OCR (latin + arabe).	
		Calculer la similarité	37.1	En tant que système, je dois calculer la similarité cosinus entre embeddings ArcFace 512-d.	Élevé
		Suggérer un prix	38.1	En tant que système, je dois suggérer un prix de livraison (XGBoost + repli heuristique).	Moyenne
		Détecter la fraude	39.1	En tant que système, je dois détecter automatiquement les avis frauduleux (XLM-RoBERTa).	Moyenne
		Suivi aérien	40.1	En tant que système, je dois interroger l'API OpenSky toutes les 30 s pour le tracking aérien.	Élevé
		Watchdog GPS	41.1	En tant que système, je dois surveiller et clore les sessions GPS abandonnées.	Moyenne
		Push notifications	42.1	En tant que système, je dois envoyer des notifications push (FCM/APNs).	Moyenne
		Webhook Stripe	43.1	En tant que système, je dois vérifier la signature HMAC SHA-256 des webhooks Stripe.	Élevé
		Diffusion temps réel	44.1	En tant que système, je dois diffuser les événements via Socket.IO (rooms user:, deal:, chat:).	Élevé

Tableau 2.1 : Backlog de produit

2.6 Planification des releases

La planification des releases constitue l'une des étapes clés d'un projet Scrum. Lors de cette réunion, l'équipe de développement identifie les éléments prioritaires du backlog produit qu'elle estime pouvoir accomplir pendant le sprint. Le résultat de cette réunion est un plan de release, élaboré avec la participation active de toute l'équipe.

Dans le cadre de notre projet Bridger, nous avons choisi d'organiser le backlog en trois releases successives en fonction de la priorité des tâches, du nombre de sprints nécessaires et de leur durée.

Release	Description	Priorité	Durée
Release 1	Analyse, conception et développement des écrans d'on-boarding, de l'inscription par OTP Twilio Verify, de la vérification biométrique	Élevé	35 jours

	KYC (téléversement pièce d'identité, capture selfie, contrôle qualité, vivacité, comparaison cosinus ArcFace), de la session JWT et de la consultation du profil utilisateur.		
Release 2	Analyse, conception et développement de la publication d'annonces (Deals expéditeur en cinq étapes, Trips voyageur), du moteur de matching ML, de la suggestion de prix XGBoost, du séquestre Stripe (deposit, escrow hold, release, refund), de la messagerie temps réel Socket.IO, du suivi multi-mode (GPS terrestre + tracking aérien OpenSky), de la confirmation de livraison par code à 6 chiffres et QR signé HMAC.	Élevé	45 jours
Release 3	Analyse, conception et développement de l'espace administrateur (dashboard React + Vite : statistiques, gestion utilisateurs, modération KYC, gestion des litiges avec SLA 72 h, modération d'avis ML, audit trail, gestion des données d'entraînement), des notifications push FCM/APNs, du système d'avis et notation, du système de litige avec preuves médias et timeline auditable, et du déploiement Azure CI/CD.	Élevé	40 jours

Tableau 2.2 : Planification des releases

2.7 Architecture en couches (Layered Architecture)

L'architecture en couches (*Layered Architecture*) — également appelée architecture en oignon — est un patron d'architecture largement utilisé dans la conception des applications back-end modernes. Il repose sur le principe de **séparation des préoccupations**, en distinguant la logique d'exposition HTTP de la logique métier et de l'accès aux données. Cette séparation facilite la répartition des tâches entre les membres de l'équipe et améliore la maintenance et l'évolution du logiciel.

Les quatre couches du modèle :

- **Couche Routage (Routes)**

La couche routage gère la déclaration des points d'entrée HTTP et leur association à des handlers. Elle ne contient aucune logique métier : elle se contente de coordonner les middlewares (authentification, validation, rate-limiting) et de déléguer le traitement aux services.

- **Couche Validation (Validators)**

La couche validation s'appuie sur des schémas Zod déclaratifs qui contrôlent la conformité des entrées (corps de requête, paramètres d'URL, paramètres de query). Toute requête malformée est rejetée avec un code 422, ce qui protège la couche métier des données invalides.

- **Couche Service (Services)**

La couche service encapsule la logique métier : règles de gestion, orchestration des opérations transactionnelles (`prisma.$transaction`), interactions avec les services externes (Stripe, Twilio, microservice IA) et émission d'événements (Socket.IO, push notifications). Elle est totalement indépendante du protocole HTTP.

- **Couche Persistance (Prisma ORM)**

La couche persistance s'appuie sur Prisma Client, qui agit comme un repository typé fortement, avec génération automatique de types TypeScript à partir du `schema.prisma`. Cette couche garantit l'intégrité référentielle et expose une API fluente pour les requêtes complexes.

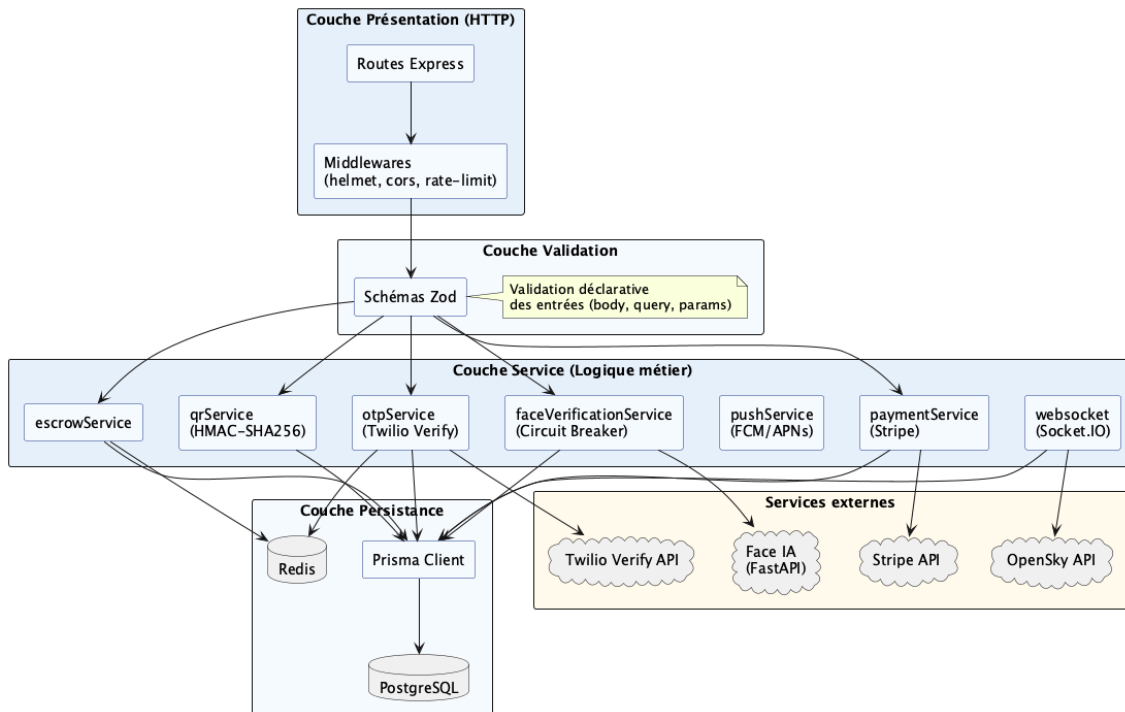


Figure 2.2 : Architecture en couches du backend Bridger

2.8 Choix architectural

Pour la partie **back-end**, nous avons opté pour le framework **Express.js** côté serveur, associé à **Prisma ORM** pour la gestion typée et la migration des données vers une base **PostgreSQL**. Le langage **TypeScript** est utilisé pour bénéficier d'un typage statique fort et d'une meilleure maintenabilité. Nous y associons **Socket.IO** pour la communication bidirectionnelle temps réel (tracking GPS, messagerie, mises à jour d'état des Deals) et **Redis** pour le cache et la gestion des verrous distribués (lockout OTP).

Pour la **vérification biométrique** et l'**intelligence artificielle**, nous avons choisi un **microservice Python** dédié, construit avec **FastAPI** et **Uvicorn**. Il intègre la bibliothèque **InsightFace** (modèles RetinaFace pour la détection et ArcFace pour l'embedding 512-d), la bibliothèque **EasyOCR** pour l'extraction multilingue des informations sur les pièces d'identité, **XGBoost** pour la suggestion de prix et **HuggingFace Transformers** (XLM-RoBERTa) pour la classification de sentiment et la détection de fraude.

Pour l'**authentification multifacteur**, nous avons choisi **Twilio Verify API** comme fournisseur principal, qui prend en charge à la fois les canaux SMS et WhatsApp. Twilio garantit la délivrabilité, masque le code à notre serveur (le code n'est connu que de Twilio jusqu'à la phase de vérification), et offre une infrastructure téléphonique mondiale conforme aux normes télécoms.

Côté **front-end mobile**, nous avons choisi **React Native** (avec **Expo SDK 54**) comme framework principal pour le développement de l'interface utilisateur cross-platform iOS / Android. React Native intègre également le client HTTP **Axios** (avec un intercepteur personnalisé pour le rafraîchissement automatique du

JWT), et utilise **Zustand** comme gestionnaire d'état global (couplé à **AsyncStorage** pour la persistance locale) et **React Navigation 7** pour la gestion des écrans et des transitions.

Côté **espace administrateur**, nous avons choisi **React** avec **Vite** comme outil de build (pour des temps de compilation rapides), **Tailwind CSS** pour le style utilitaire, et **Recharts** pour les visualisations analytiques. La SPA admin est déployée sur **Azure Static Web Apps** via GitHub Actions.

Les **paiements** sont intégrés via **Stripe** et **Stripe Connect** (pour les payouts vers les voyageurs), avec une vérification systématique des signatures de webhook.

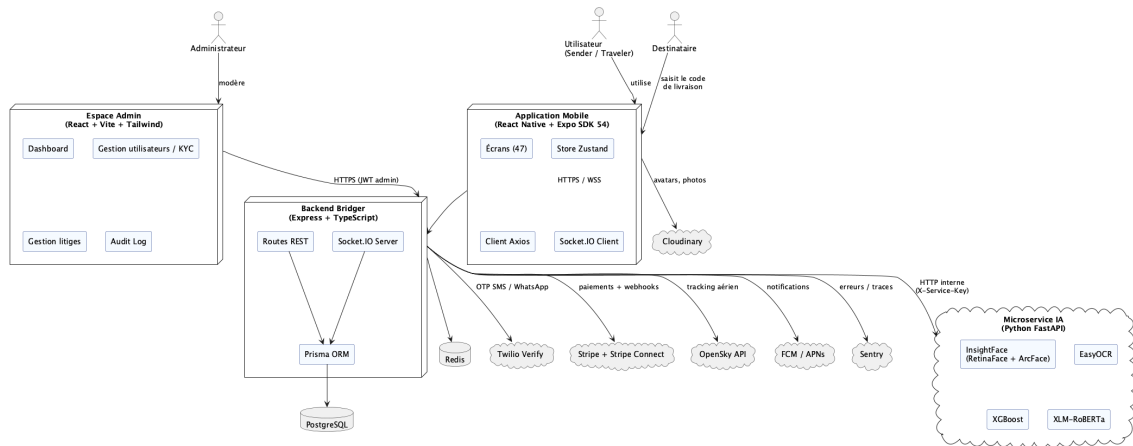


Figure 2.3 : Architecture globale de la plateforme Bridger

2.9 Environnement de travail

Dans cette partie, nous présentons les différents outils matériels et logiciels, les langages de programmation et de modélisation, ainsi que le choix technique utilisé pour mettre en œuvre notre projet.

2.9.1 Environnement matériel

Nous commençons par répertorier les caractéristiques essentielles de l'environnement matériel nécessaires à la réalisation du projet dans le tableau ci-dessous.

Nom du matériel	Mémoire RAM	Processeur	Carte graphique
MacBook Pro M-series	16 Go	Apple Silicon (ARM64)	GPU intégré Apple
HP Windows 11	16 Go	11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80 GHz	Intel Iris Xe
Smartphone Android (test)	8 Go	Snapdragon 8 Gen 1	Adreno 730
iPhone (test)	6 Go	Apple A15 Bionic	Apple GPU 5-core

Tableau 2.3 : Environnement matériel

2.9.2 Environnement logiciel

Dans cette section, on présente l'environnement technologique en spécifiant les outils logiciels, les frameworks et les langages de programmation utilisés pour développer notre projet.

Logo	Définition
VS Code	Visual Studio Code est un éditeur de code extensible développé par Microsoft pour Windows, Linux et macOS. C'est l'IDE principal utilisé pour le développement TypeScript / JavaScript / Python.
Expo	Expo est un toolchain et une plateforme cloud bâtie autour de React Native, permettant de développer, tester et déployer des applications mobiles sans configuration native complexe. Le SDK 54 a été utilisé.
Figma	Figma est un éditeur de graphiques vectoriels et un outil de prototypage. Il est principalement basé sur le web, utilisé pour la conception des maquettes de l'application mobile et de l'espace admin.
Git	Git est un système de contrôle de version distribué inventé et développé par Linus Torvalds. Il aide une équipe de développeurs à gérer les changements apportés au code source au fil du temps.
Postman	Postman est un outil qui permet aux développeurs de tester et de documenter les API REST. Il permet de créer des requêtes HTTP, de gérer les paramètres et les en-têtes, de vérifier les réponses et les statuts de retour, et de sauvegarder des collections de requêtes.
React Native	React Native est un framework open-source créé par Meta pour développer des applications mobiles attrayantes, rapides et multi-plateformes (iOS / Android) à partir d'une seule base de code. La version 0.81 a été utilisée.
Express.js	Express.js est un framework web minimaliste et flexible pour Node.js, utilisé pour bâtir des API REST robustes. Il fournit une couche fine au-dessus du module HTTP natif et un système de middlewares puissant.
Prisma ORM	Prisma est un ORM moderne pour Node.js et TypeScript qui génère automatiquement un client typé fortement à partir d'un schéma déclaratif. Il prend en charge PostgreSQL, MySQL, SQLite et SQL Server.
TypeScript	TypeScript est un langage de programmation libre développé par Microsoft qui a pour but d'améliorer et de sécuriser la production de code JavaScript par l'ajout du typage statique.
React + Vite	React est une bibliothèque JavaScript développée par Meta pour créer des interfaces utilisateur. Vite est un outil de build moderne et ultra-rapide qui sert le code source via des modules ES natifs en développement. Utilisé pour l'espace administrateur.
Node.js	Node.js est un environnement d'exécution JavaScript libre, orienté vers les applications réseau évènementielles hautement concurrentes qui doivent pouvoir monter en charge. La version 22 LTS a été utilisée.
Tailwind CSS	Tailwind CSS est un framework CSS open-source utility-first. Il fournit une vaste collection de classes utilitaires permettant de composer rapidement des interfaces

	sans quitter le HTML/JSEX.
PostgreSQL	PostgreSQL est un puissant système de gestion de base de données relationnelle open source. Il prend en charge l'extension pgvector pour la recherche vectorielle ANN, prévue pour le stockage des embeddings biométriques.
Redis	Redis est un magasin de structures de données en mémoire, utilisé comme cache distribué, gestionnaire de verrous (lockout OTP) et bus de messages.
FastAPI	FastAPI est un framework web Python moderne, performant et asynchrone, basé sur Starlette et Pydantic. Il est utilisé pour le microservice de vérification biométrique et d'intelligence artificielle.
InsightFace	InsightFace est une bibliothèque Python open-source dédiée à la détection et à la reconnaissance faciale. Elle intègre les modèles RetinaFace (détection) et ArcFace (embedding 512-d).
EasyOCR	EasyOCR est une bibliothèque Python d'OCR multilingue (plus de 80 langues, dont le latin et l'arabe). Elle est utilisée pour extraire automatiquement le numéro et la date de naissance des pièces d'identité.
XGBoost	XGBoost (Extreme Gradient Boosting) est une bibliothèque ML hautement performante pour les arbres de décision boostés. Elle est utilisée pour la suggestion de prix de livraison.
Stripe	Stripe est une plateforme de paiement en ligne. Bridger utilise Stripe pour la collecte des paiements, le séquestre (escrow), Stripe Connect pour les payouts vers les voyageurs, et la vérification des webhooks.
Twilio	Twilio est une plateforme de communication cloud. Bridger utilise Twilio Verify API pour l'envoi et la vérification des codes OTP via SMS ou WhatsApp, garantissant la délivrabilité et la sécurité du canal.

Tableau 2.4 : Environnement logiciel

2.9.3 Plateformes utilisées

Logo	Définition
GitHub	GitHub est une plateforme de gestion de code source qui permet aux développeurs de stocker, partager et collaborer sur des projets de programmation. Elle utilise Git, un système de contrôle de version, pour faciliter la gestion des modifications. GitHub Actions est utilisé pour le pipeline CI/CD.
Slack	Slack est une application de messagerie pour les entreprises qui connecte les personnes aux informations dont elles ont besoin. Utilisée pour la communication d'équipe pendant le projet.
Visual Paradigm Online	Visual Paradigm Online est un outil de création de diagrammes basé sur le Web qui prend en charge un grand nombre de diagrammes commerciaux et techniques. Utilisé pour les premières esquisses des diagrammes UML.

Swagger / OpenAPI	Swagger est un langage de description d'interface permettant de décrire des API exprimées à l'aide de JSON. Utilisé pour documenter les routes REST de l'API Bridger.
Azure App Service	Microsoft Azure App Service est un service de cloud computing PaaS pour héberger des applications web. Il héberge le backend Express et le microservice IA.
Cloudinary	Cloudinary est une plateforme cloud de gestion des médias. Elle est utilisée pour l'hébergement et la transformation des avatars utilisateurs et des photos de colis.
Sentry	Sentry est une plateforme open-source de suivi des erreurs. Elle capture les exceptions du backend et du mobile en production avec un échantillonnage des traces (tracesSampleRate: 0.2).

Tableau 2.5 : Plateformes utilisées

2.10 Écosystème du projet

Dans cette section, nous présentons les différents outils, technologies et frameworks qui composent l'écosystème technique de notre application. Le projet repose sur un **front-end mobile** développé avec **React Native (Expo SDK 54)**, permettant une expérience utilisateur fluide et multiplateforme. Pour la communication avec le serveur, nous utilisons le client HTTP **Axios**, connu pour sa gestion efficace des requêtes et des erreurs (intercepteur de rafraîchissement JWT, gestion fine des codes d'erreur). Le **back-end** est conçu à l'aide du framework **Express.js**, robuste et largement adopté, tandis que la gestion des données est assurée par **PostgreSQL** via **Prisma ORM**. Une **base Redis** complète l'architecture pour le cache et les verrous distribués.

L'**intelligence artificielle** est isolée dans un microservice Python dédié construit avec **FastAPI**, embarquant les modèles **InsightFace** (RetinaFace + ArcFace), **EasyOCR**, **XGBoost** et **XLM-RoBERTa**. L'**authentification** repose sur **Twilio Verify API** pour les codes OTP (SMS / WhatsApp). Les **paiements** sont gérés par **Stripe** et **Stripe Connect**. L'**espace administrateur** est une SPA **React + Vite + Tailwind CSS** déployée sur Azure Static Web Apps. Cette combinaison de technologies nous permet de bâtir une architecture robuste, évolutive et moderne.

 Figure 2.4 — Schéma de l'écosystème de projet Bridger

Figure 2.4 : Schéma de l'écosystème de projet Bridger

2.10.1 Express.js et Prisma ORM

Dans le cadre du développement de l'application **Bridger**, une plateforme pair-à-pair de livraison collaborative, le choix d'une architecture backend adaptée était essentiel pour répondre aux exigences de complexité fonctionnelle, de performance, de sécurité et de scalabilité.

Nous avons opté pour **Express.js** (framework web minimaliste pour Node.js, en TypeScript) et **Prisma ORM** pour la gestion typée et maintenable des données.

Express.js nous permet de structurer des API REST modernes pour gérer l'ensemble des fonctionnalités : utilisateurs, deals, trips, messagerie, contrats de paiement, séquestre, litiges, etc. Grâce à son architecture en middlewares, nous avons facilement intégré la gestion de l'authentification JWT, des rôles (`isAdmin`), de la limitation de débit (`express-rate-limit`), du durcissement HTTP (`helmet`) et de la validation déclarative des entrées (Zod).

Prisma ORM s'intègre naturellement avec Express, en nous offrant une modélisation claire, typée et optimisée des entités (`User` , `Deal` , `Trip` , `Transaction` , `Dispute` , `ChatRoom` , `FaceScan` , `KycDocument` , `AuditLog` , etc.). Il facilite la génération de requêtes SQL fiables, supporte les transactions atomiques (`prisma.$transaction`) indispensables pour la cohérence du séquestre, et sépare proprement la logique métier de la couche base de données.

Ce duo technologique couvre aussi les besoins non fonctionnels clés :

- **Performance** (API rapides, indexation Prisma fine, cache Redis, lazy-loading des modèles ML),
- **Sécurité** (JWT signés, middlewares de validation, vérification HMAC, audit trail immuable),
- **Scalabilité** (architecture microservices conteneurisable, watchdog pour les tâches longue durée),
- **Multiplateforme** (React Native pour iOS / Android, React + Vite pour l'admin web),
- **Confidentialité** (uploads hors arborescence publique, codes QR signés, conformité RGPD).

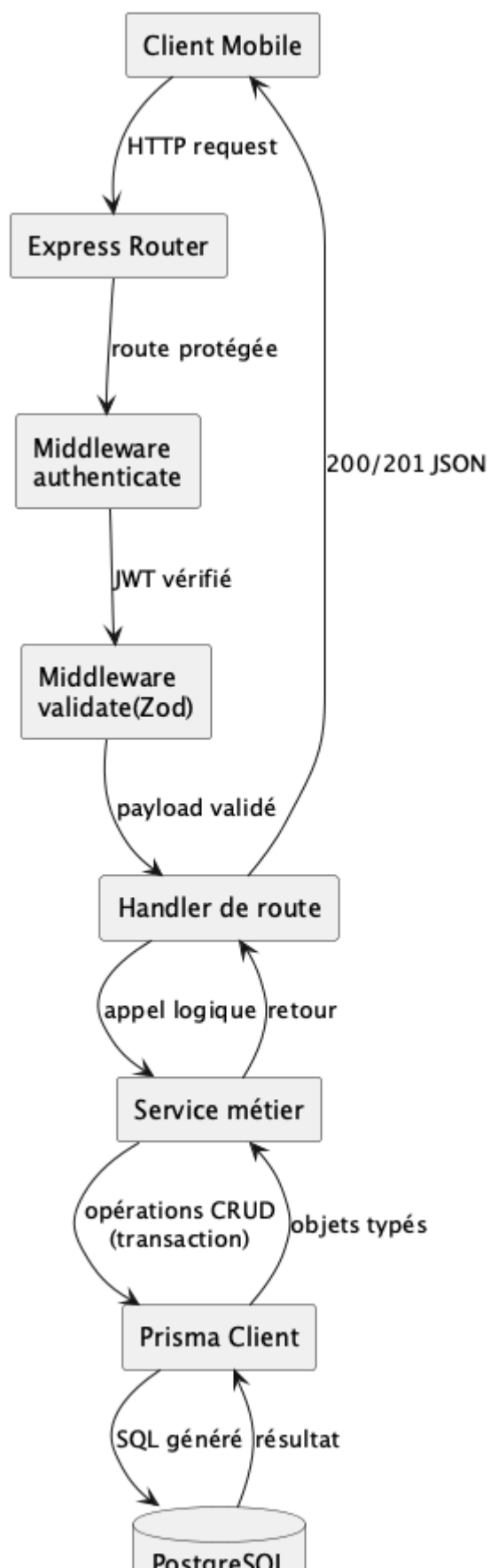




Figure 2.5 : Schéma de fonctionnement d'Express.js et Prisma ORM dans l'application

2.10.2 PostgreSQL et Twilio Verify

PostgreSQL est un système de gestion de base de données relationnel open-source, robuste, conforme à ACID et largement adopté dans l'industrie. Il propose nativement les fonctions transactionnelles, les index B-tree et hash, le typage fort, les contraintes d'intégrité référentielle, et — par le biais de l'extension `pgvector` — la recherche vectorielle ANN (Approximate Nearest Neighbour) pour les embeddings biométriques. C'est dans ce contexte que nous avons choisi PostgreSQL pour répondre aux exigences de l'application Bridger en matière de cohérence transactionnelle (séquestre Stripe), de performance (index composites sur les colonnes filtrées) et de scalabilité. Son intégration fluide avec notre backend Express.js et notre ORM Prisma facilite la modélisation typée et maintenable des 26 entités du domaine. PostgreSQL garantit également la conformité RGPD via la cascade `onDelete: Cascade` qui propage la suppression d'un utilisateur à toutes ses données dérivées.

Twilio Verify API est un service cloud d'authentification multifacteur opéré par **Twilio**, leader mondial des plateformes de communication. Il permet d'envoyer des codes OTP via SMS, WhatsApp ou voix, et de vérifier le code saisi par l'utilisateur sans que le serveur applicatif ne connaisse jamais le code en clair. Cette propriété — souvent désignée *zero-knowledge OTP* — élimine une classe entière de vulnérabilités (fuite de logs, compromission de la base de données). Twilio garantit également la conformité aux normes télécoms, la délivrabilité internationale et la résistance aux attaques par numérotation premium frauduleuse. Dans le cadre du projet Bridger, Twilio Verify est configuré avec deux canaux complémentaires : **SMS** (universel) et **WhatsApp** (pour les marchés où WhatsApp est dominant). Une logique de repli (DB-based) couvre les cas où Twilio est indisponible.

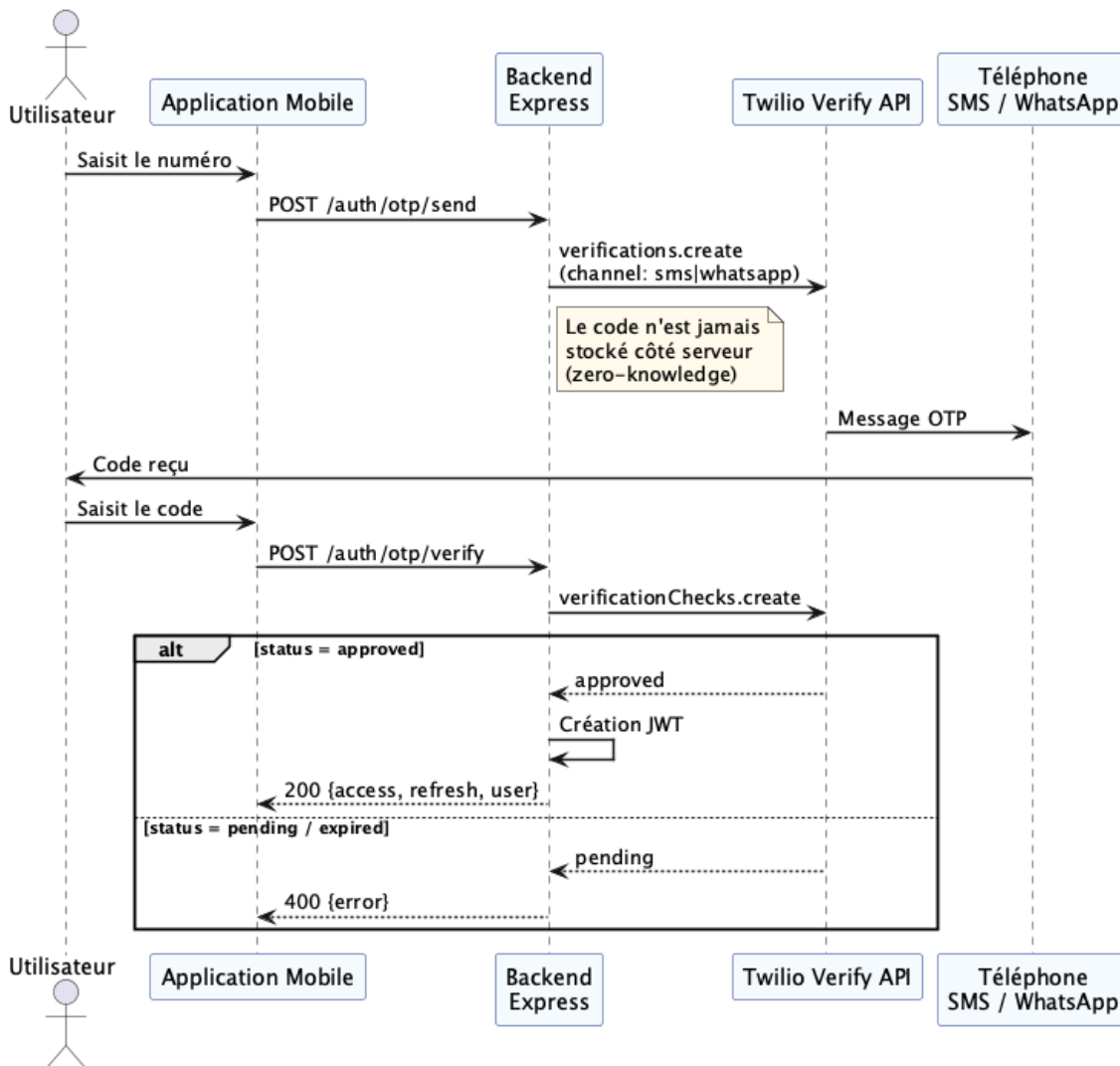


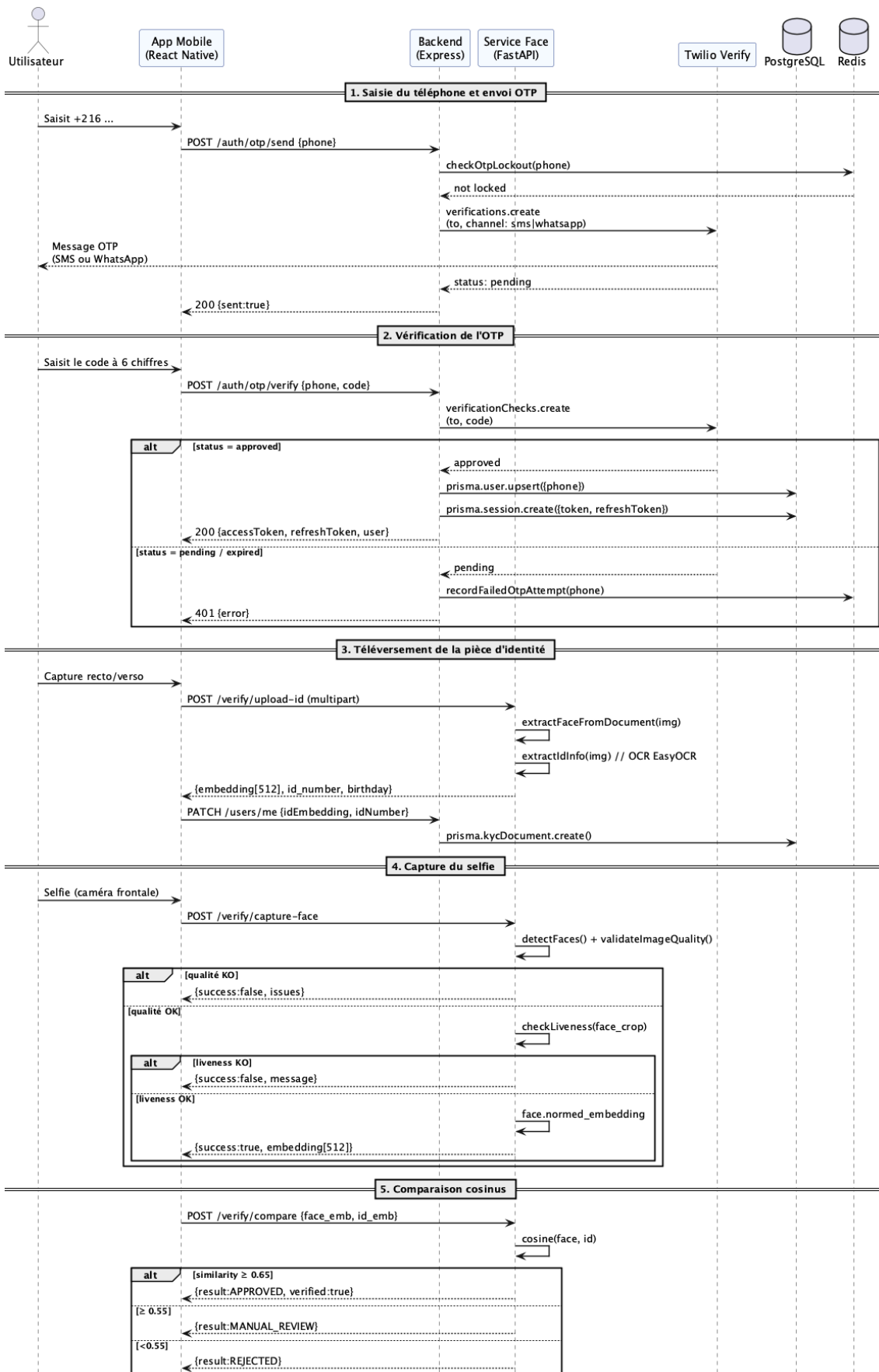
Figure 2.6 : Schéma de fonctionnement de Twilio Verify dans le flux d'authentification

2.11 Modélisation détaillée du système

En complément du diagramme de cas d'utilisation présenté en section 2.4, cette section détaille la **structure statique** (diagramme de classes), le **comportement dynamique** (diagrammes de séquence) et le **cycle de vie** (diagramme d'état) des entités les plus critiques du système.

2.11.1 Diagramme de classes

Le diagramme de classes formalise la structure statique du domaine, dérivée du fichier `backend/prisma/schema.prisma`. Il distingue les visibilités, les multiplicités et les natures d'association : composition (`*--`) pour les agrégats forts matérialisés par `onDelete: Cascade`, agrégation (`o--`) pour les liens faibles, association simple (`-->`) pour les références croisées.



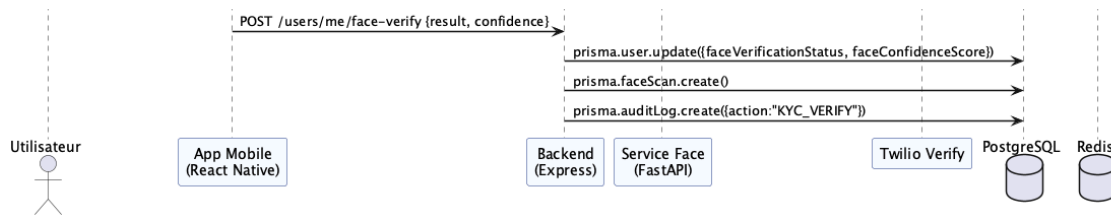


Figure 2.8 : Diagramme de séquence — Inscription et vérification KYC biométrique

Description : Le système de seuils à trois niveaux — APPROVED ($\geq 0,65$), MANUAL_REVIEW (0,55–0,65), REJECTED ($< 0,55$) — permet un compromis pragmatique entre faux rejets (FRR) et faux acceptations (FAR) : les cas ambigus sont escaladés vers la modération humaine plutôt que rejetés frontalement. L' AuditLog final fixe une preuve immuable pour la conformité RGPD.

2.11.3 Diagramme de séquence — Publication d'un Deal, appariement et séquestre

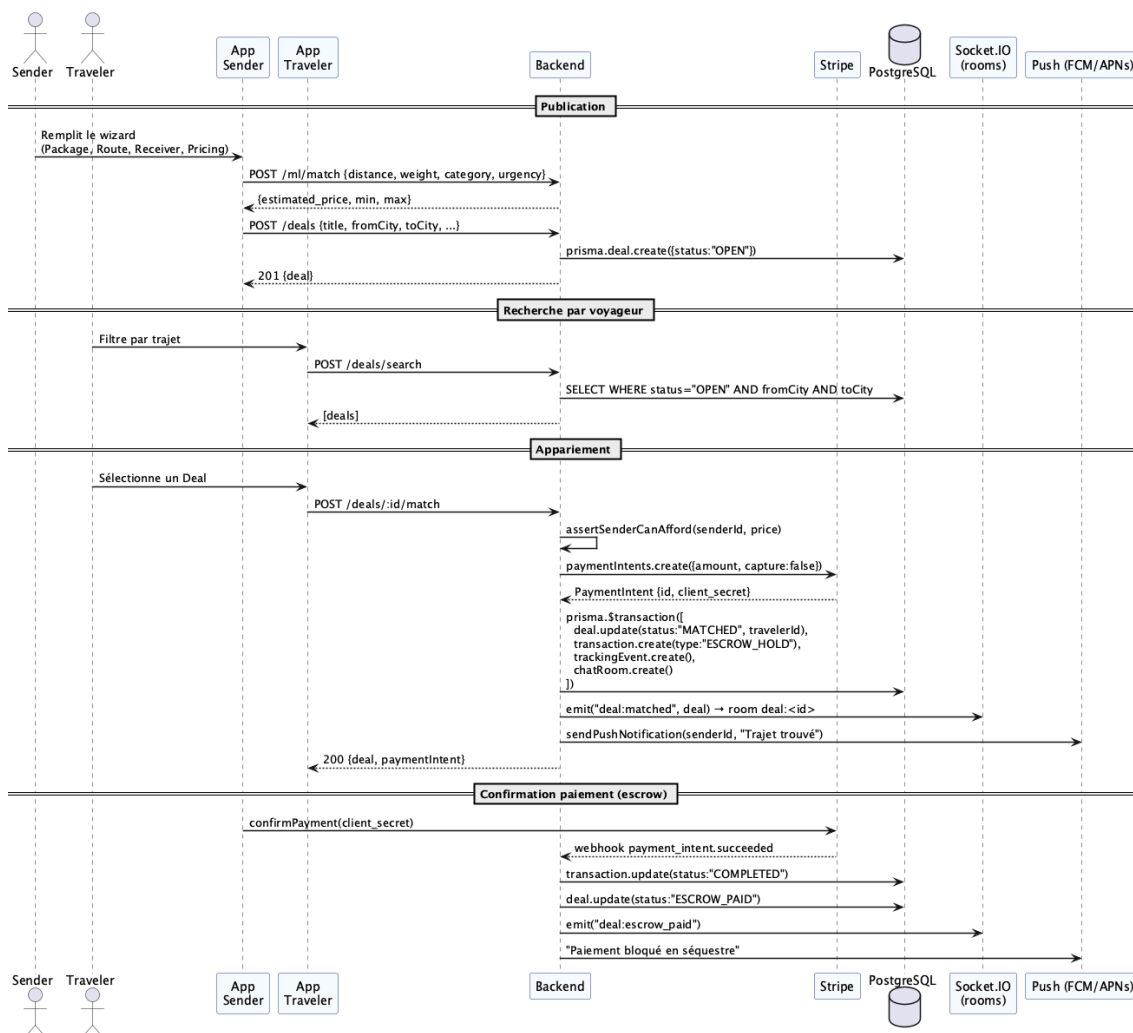


Figure 2.9 : Diagramme de séquence — Publication d'un Deal, appariement et séquestre

Description : L'appariement déclenche en transaction atomique (`prisma.$transaction`) la mise à jour du Deal, la création de la transaction d' `ESCROW_HOLD` , la création du premier `TrackingEvent` et l'ouverture

de la `ChatRoom`. La diffusion temps réel via `Socket.IO` informe immédiatement les deux parties, complétée par une notification push pour les utilisateurs hors application.

2.11.4 Diagramme de séquence — Suivi multi-mode et libération du séquestre

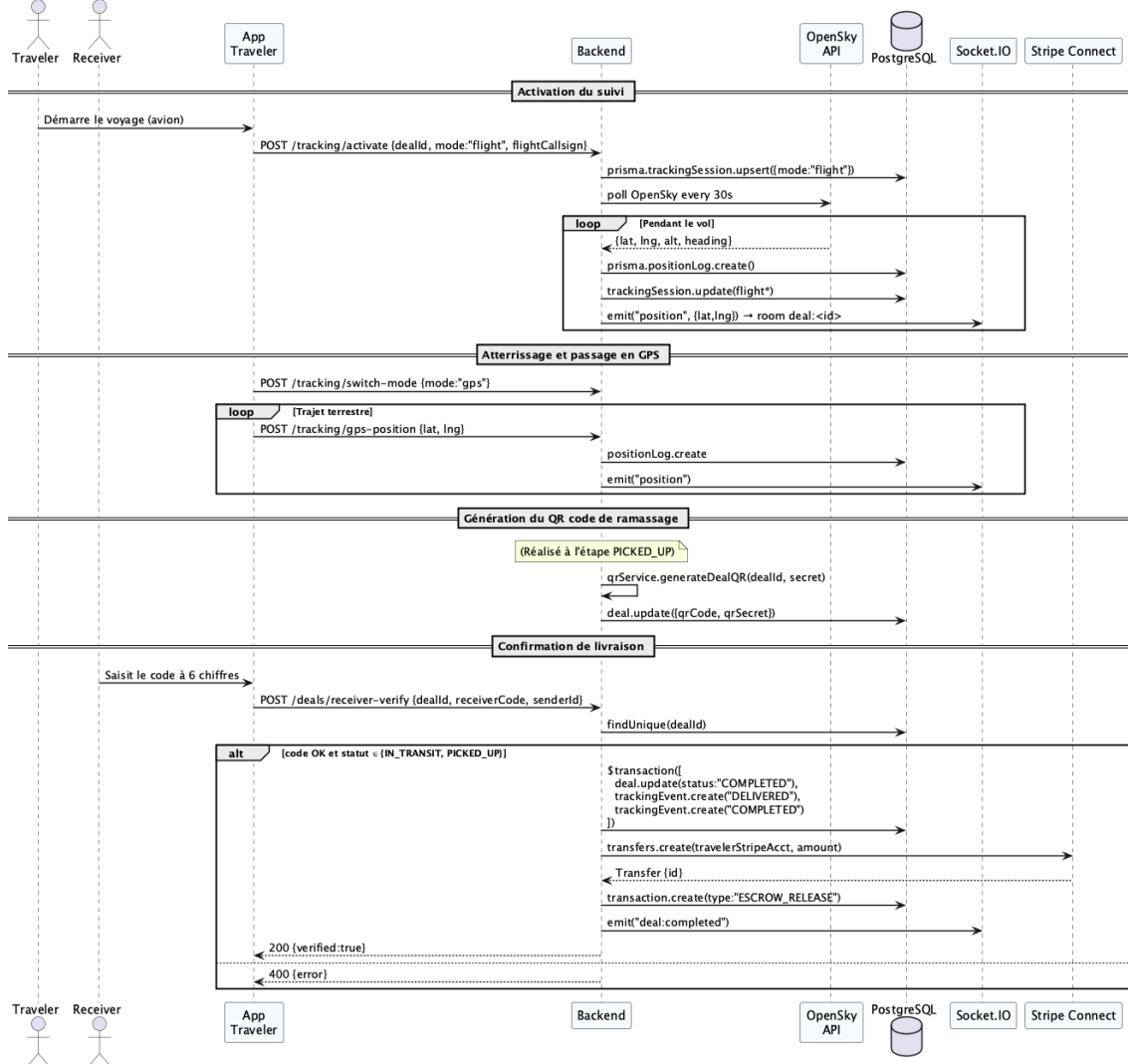


Figure 2.10 : Diagramme de séquence — Suivi multi-mode et libération du séquestre

Description : Le suivi bascule automatiquement entre les modes `flight` (polling `OpenSky` toutes les 30 s) et `gps` (positions émises par l'application toutes les 10 s). Toutes les positions sont persistées dans `PositionLog` pour permettre le replay du trajet. La confirmation de livraison par le destinataire (saisie d'un code à 6 chiffres) déclenche en transaction atomique le passage du Deal en `COMPLETED`, l'ajout des étapes manquantes dans la timeline et le transfert `Stripe Connect` vers le voyageur.

2.11.5 Diagramme de séquence — Ouverture et résolution d'un litige

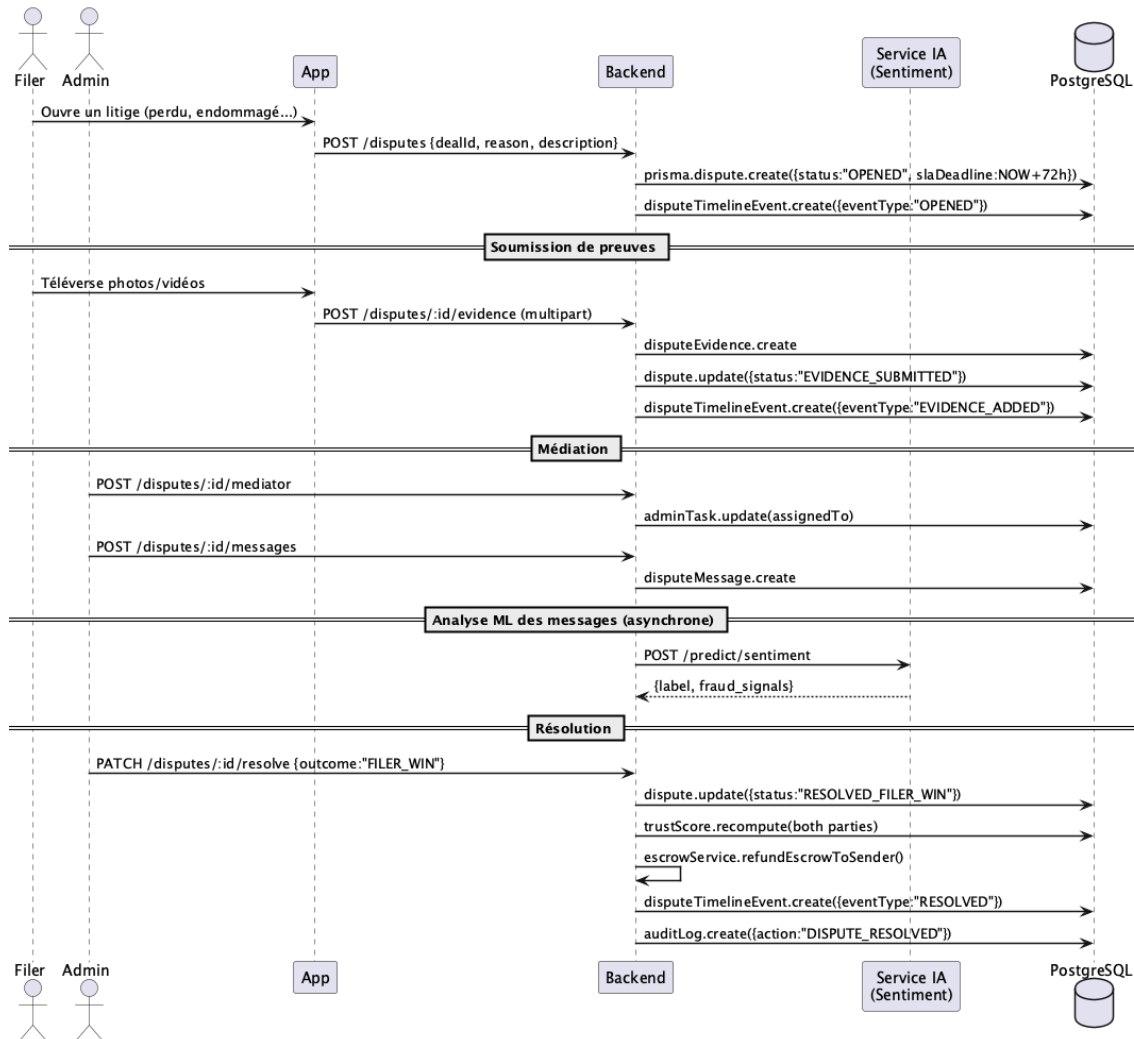


Figure 2.11 : Diagramme de séquence — Ouverture et résolution d'un litige

Description : Le litige est régi par une **SLA de 72 heures** (`slaDeadline = NOW() + interval '72 hours'`). Chaque transition d'état alimente la `DisputeTimelineEvent` , ce qui permet à l'interface de rendre la chronologie sans recalcul. La résolution déclenche soit un remboursement automatique vers l'expéditeur (`FILER_WIN`), soit la libération différée du séquestre vers le voyageur (`AGAINST_WIN`), garantissant un règlement déterministe.

2.11.6 Diagramme d'état — Cycle de vie d'un Deal

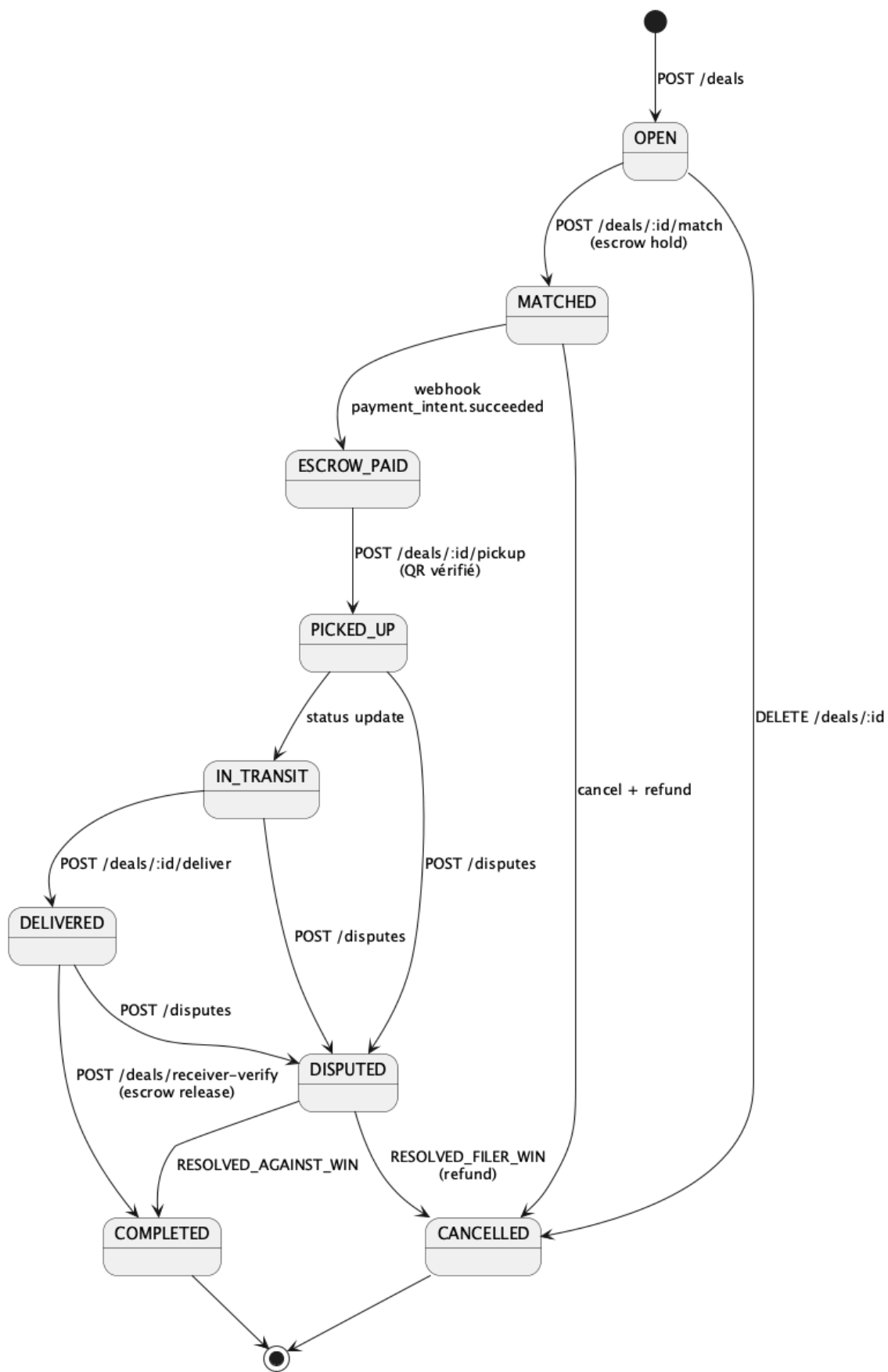


Figure 2.12 : Diagramme d'état — Cycle de vie d'un Deal

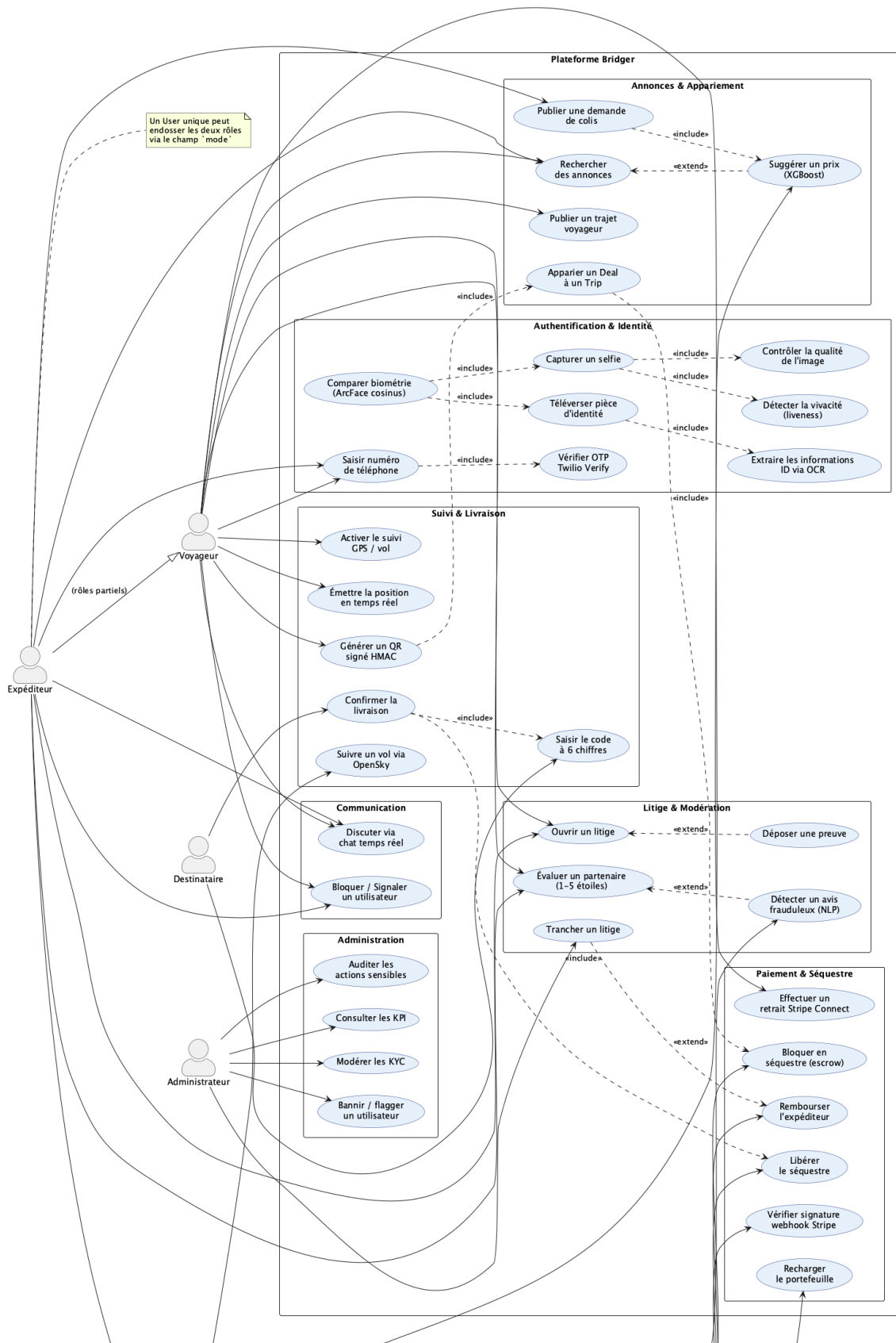
Description : L'analyse des routes `/deals/:id/match`, `/deals/:id/pickup`, `/deals/:id/deliver`, `/deals/receiver-verify` et `/disputes` permet de reconstituer la machine à états du Deal. Cette machine est **invariante** : toute transition qui n'apparaît pas explicitement est rejetée par les handlers avec un code 400, conformément au pattern défensif visible à la ligne `if (!['IN_TRANSIT', 'PICKED_UP', 'ESCROW_PAID'].includes(deal.status))`.

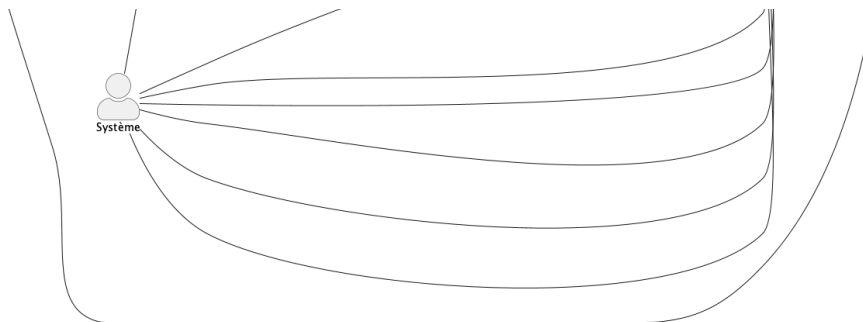
Conclusion

Dans ce deuxième chapitre, nous avons défini les besoins fonctionnels et non fonctionnels de la plateforme **Bridger**, identifié les acteurs (Expéditeur, Voyageur, Destinataire, Administrateur, Système), élaboré le backlog produit avec une priorisation par release, et formalisé l'architecture en couches retenue ainsi que l'écosystème technologique (React Native, Express.js, Prisma, PostgreSQL, Redis, FastAPI + InsightFace, Twilio Verify, Stripe, React + Vite). Nous avons enfin présenté la modélisation UML détaillée — diagramme de cas d'utilisation, diagramme de classes, diagrammes de séquence, diagramme d'état — qui formalise respectivement les frontières fonctionnelles, la structure statique, le comportement dynamique et le cycle de vie des entités centrales. Dans le chapitre suivant, nous exposerons en détail la conception et la réalisation de la première release de l'application.

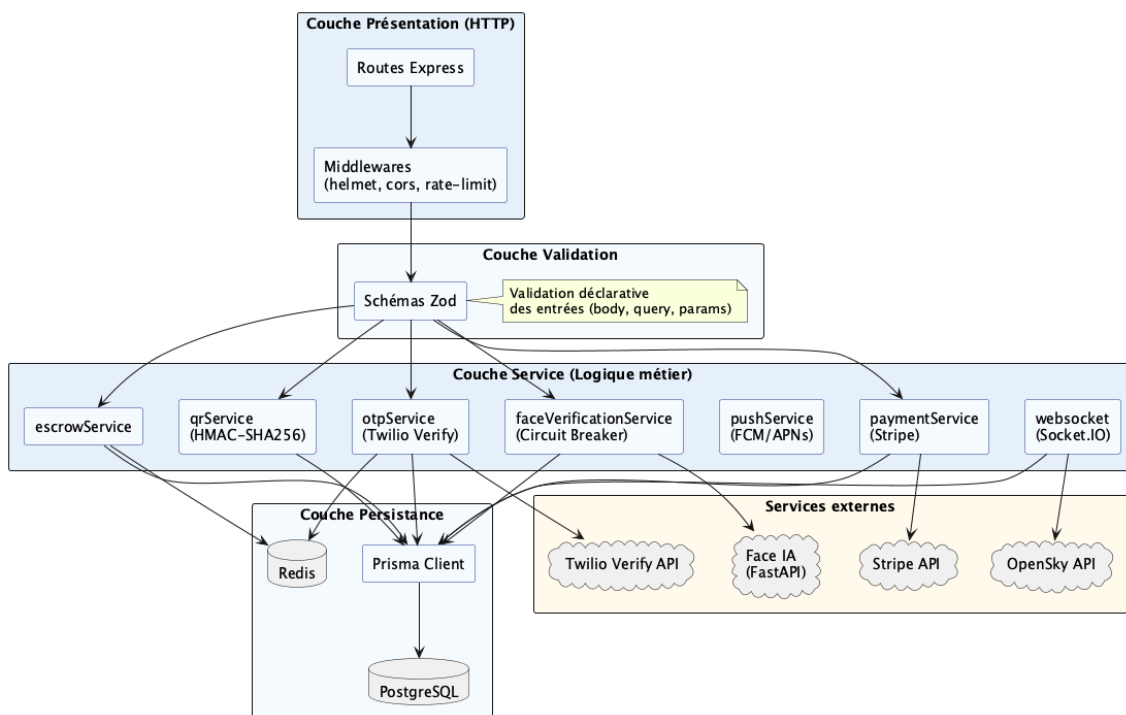
Annexe A — Sources des diagrammes UML (PlantUML)

A.1 Diagramme de cas d'utilisation

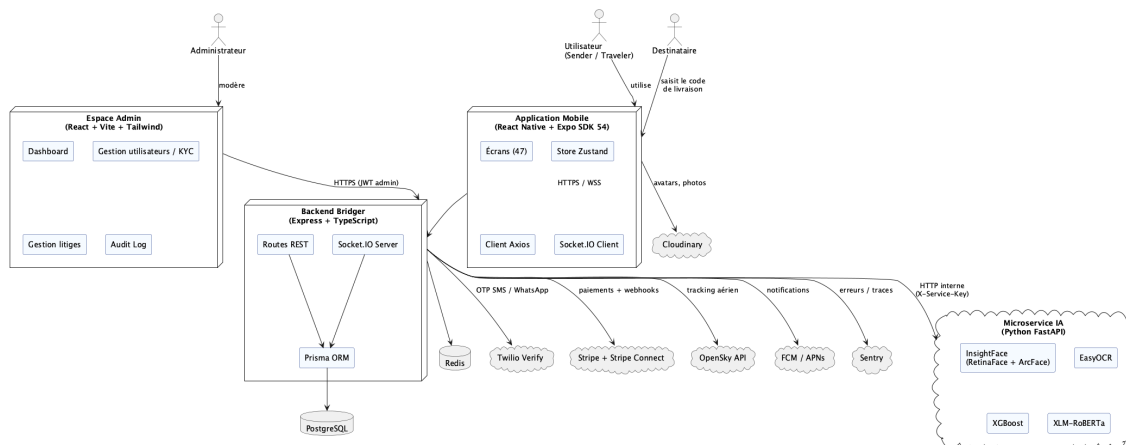




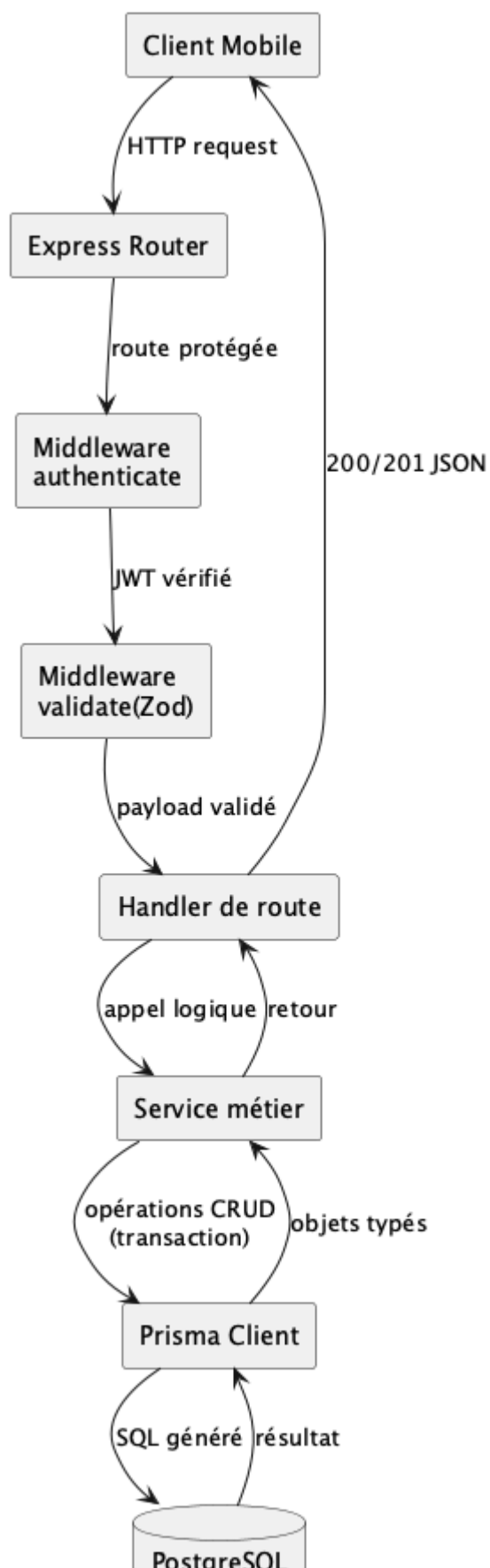
A.2 Diagramme d'architecture en couches



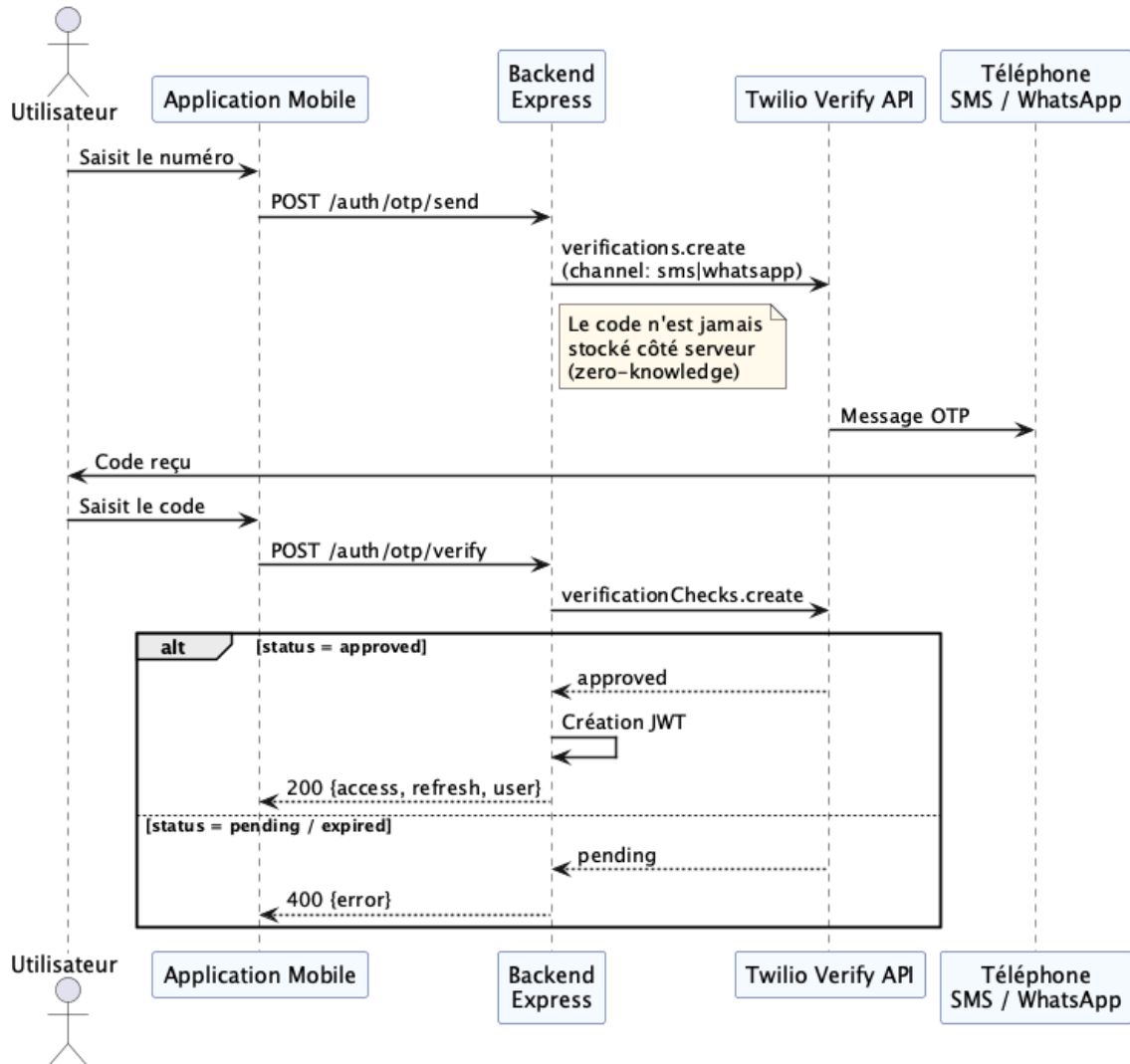
A.3 Diagramme d'architecture globale



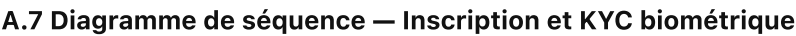
A.4 Diagramme de fonctionnement Express + Prisma

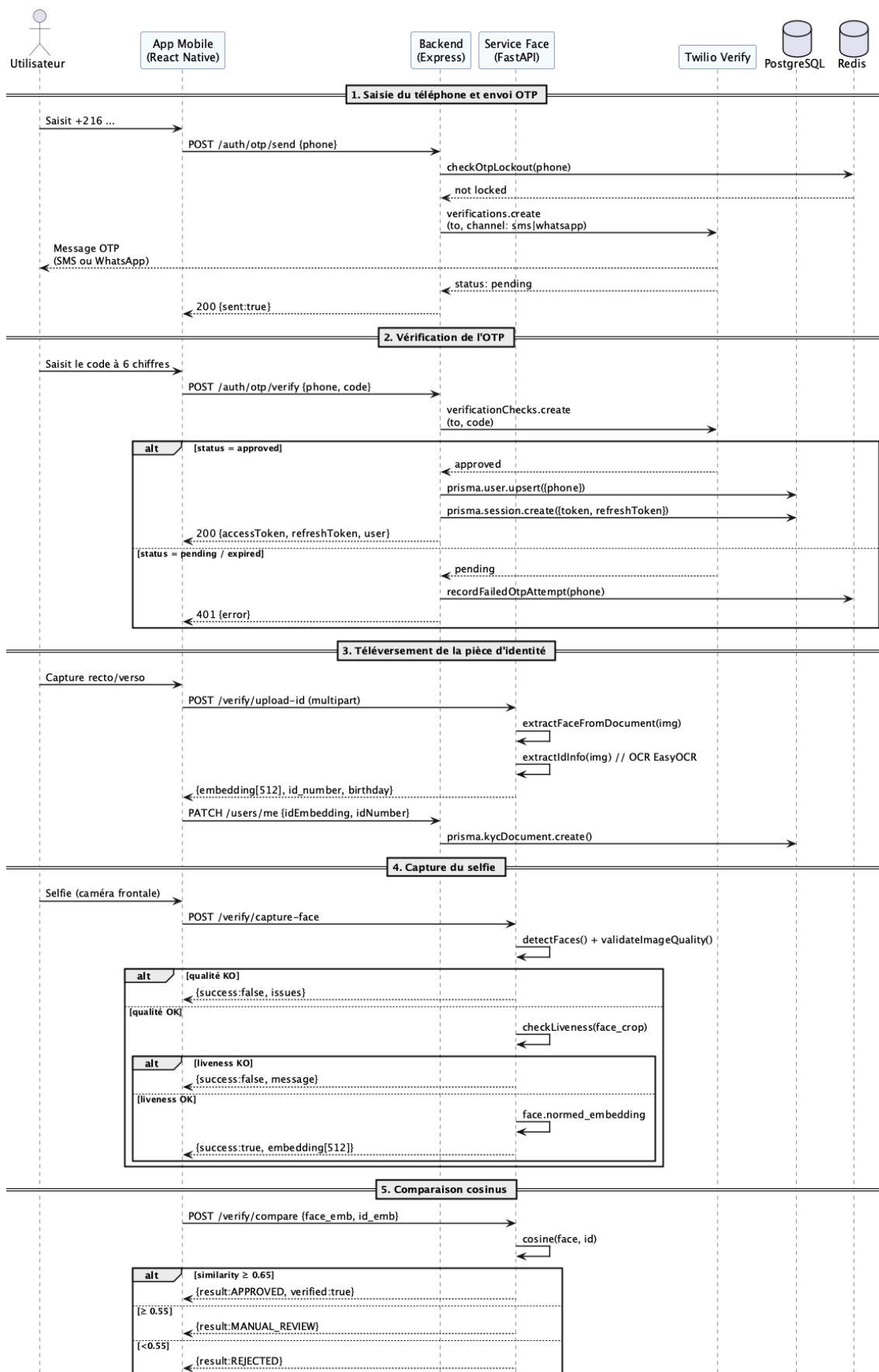


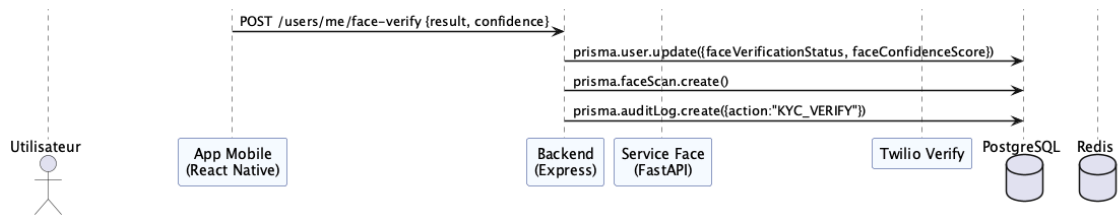
A.5 Diagramme de fonctionnement Twilio Verify



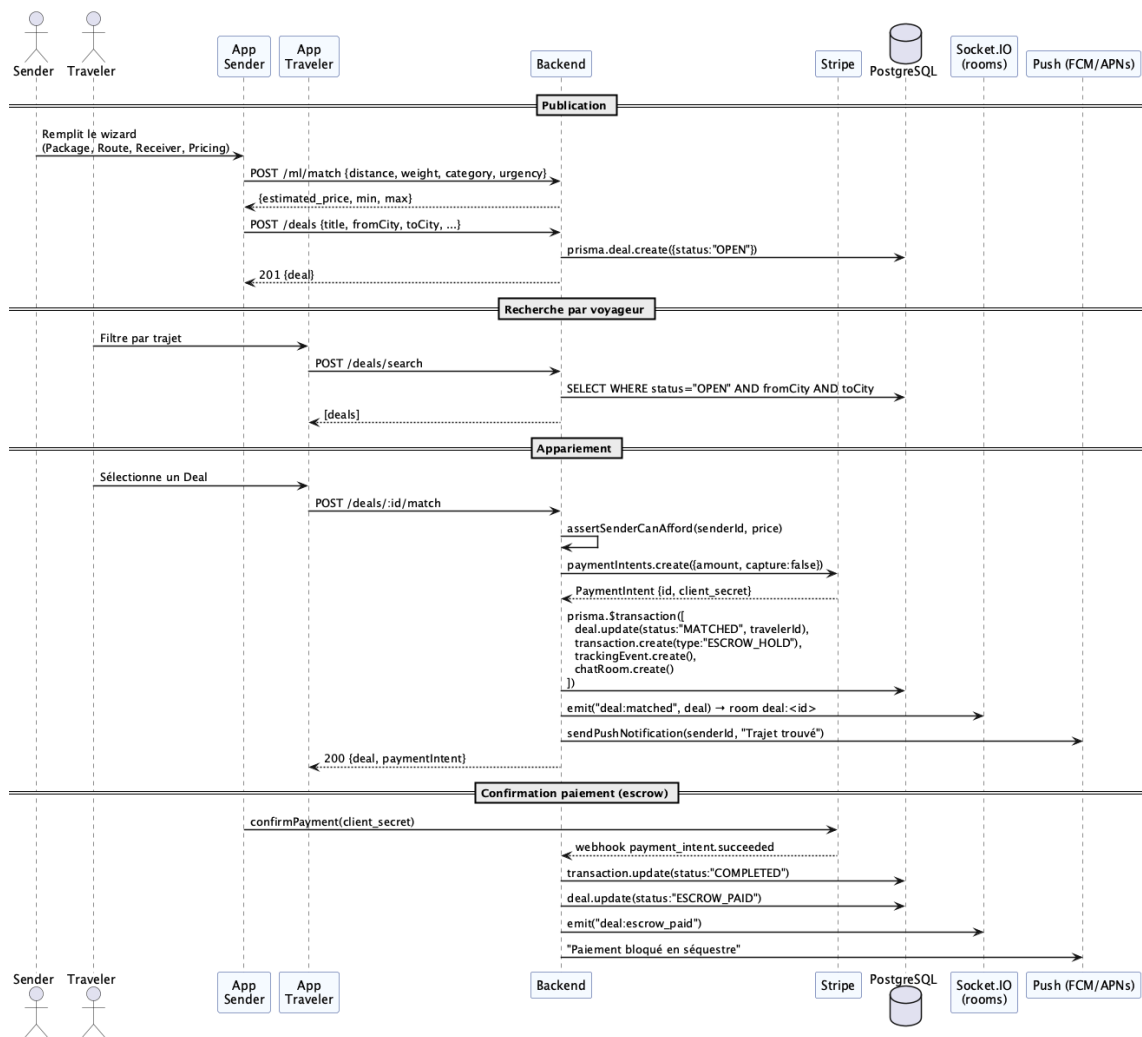
A.6 Diagramme de classes



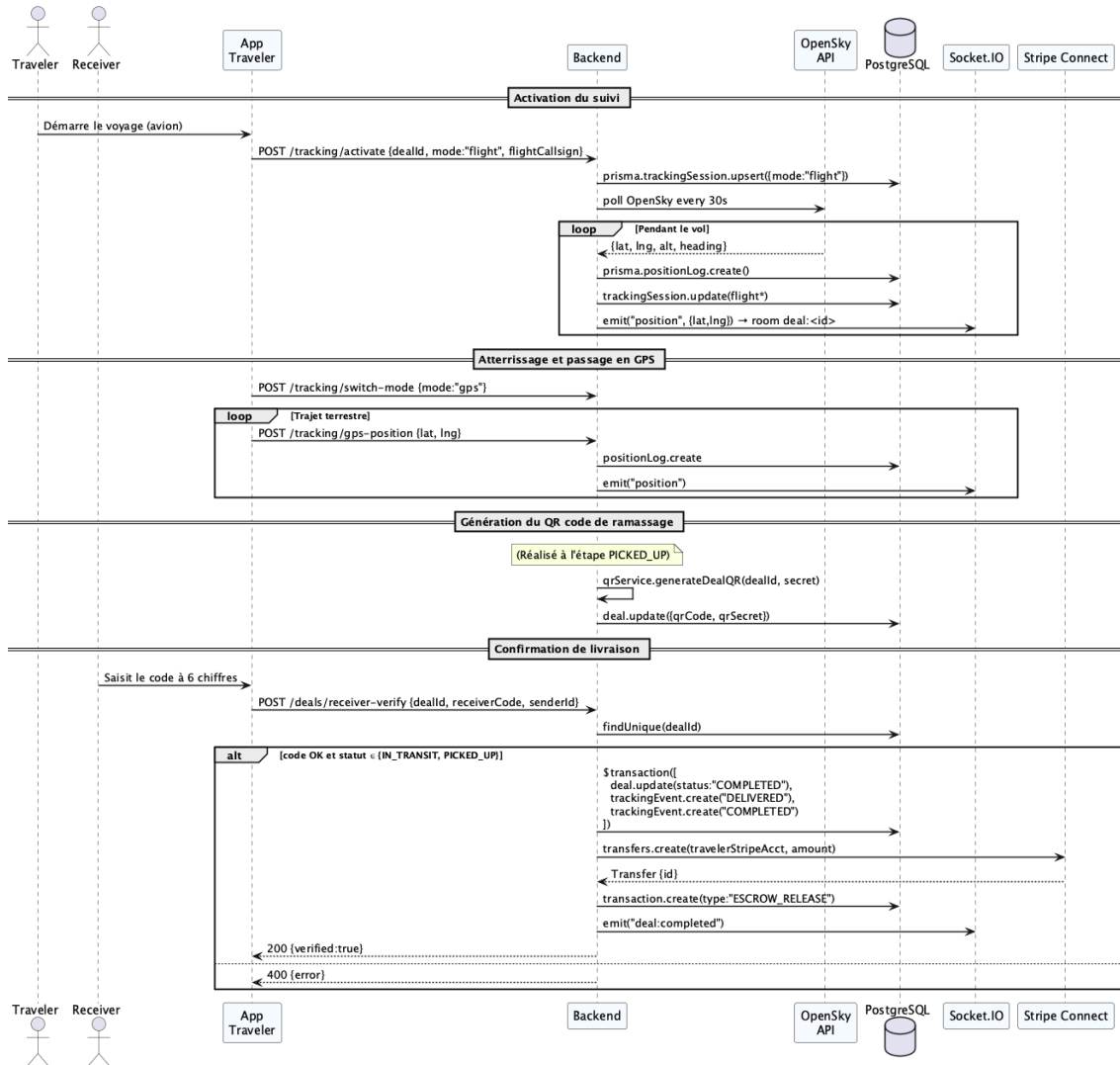




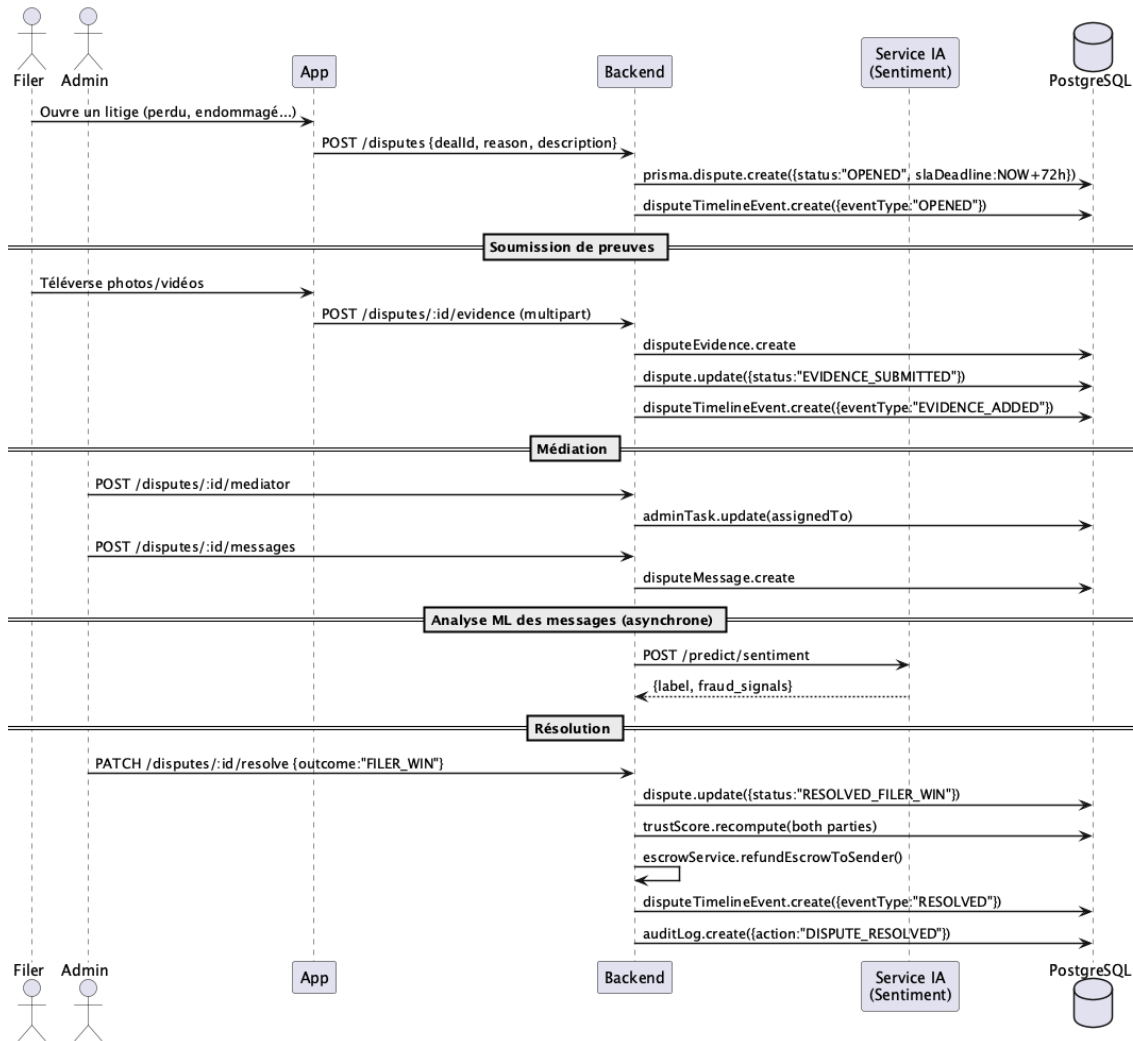
A.8 Diagramme de séquence — Publication, matching et escrow



A.9 Diagramme de séquence — Suivi GPS et libération



A.10 Diagramme de séquence — Litige



A.11 Diagramme d'état du Deal

